A Project Report on

Sentiment-Augmented Stock Price Forecasting

Submitted in partial fulfillment of the requirements for the award
of the degree of

Bachelor of Engineering

in

Computer Science and Engineering - Data Science

by

Sumit Shahu(21107004)
Pravesh Yadav(21107057)
Mohd Mustafa Shaikh(21107045)
Ankit Purohit(21107020)

Under the Guidance of

Ms.Ujwala Pagare
Ms.Aavani Nair

Department of Computer Science and Engineering - Data Science
A.P. Shah Institute of Technology
G.B.Road,Kasarvadavli, Thane(W)-400615
UNIVERSITY OF MUMBAI

Academic Year 2024-2025

Approval Sheet

This Project Report entitled ***"Sentiment-Augmented Stock Price Forecasting"*** Submitted by ***"Sumit Shahu"(21107004), "Pravesh Yadav"(21107057), " Mohd Mustafa Shaikh"(21107045), "Ankit Purohit"(21107020)***is approved for the partial fulfillment of the requirement for the award of the degree of ***Bachelor of Engineering*** in ***Computer Science and Engineering - Data Science*** from ***University of Mumbai***.

Ms.Aavani Nair
Co-Guide

Ms.Ujwala Pagare
Guide

Ms. Anagha Aher
HOD, Computer Science and Engineering - Data Science

Place:A.P.Shah Institute of Technology, Thane
Date:

CERTIFICATE

This is to certify that the project entitled ***"Sentiment-Augmented Stock Price Forecasting"***submitted by ***" Sumit shahu" (21107004),"Pravesh Yadav "(21107057)," Mohd Mustafa Shaikh" (21107045),"Ankit Purohit" (21107020)*** for the partial fulfillment of the requirement for award of a degree ***Bachelor of Engineering*** in ***Computer Science and Engineering - Data Science***,to the University of Mumbai,is a bonafide work carried out during academic year 2023-2024.

Ms.Aavani Nair
Co-Guide

Ms.Ujwala Pagare
Guide

Ms. Anagha Aher
HOD, CSE(Data Science)

Dr. Uttam D.Kolekar
Principal

External Examiner(s)

1.

2.

Internal Examiner(s)

1.

2.

Place:A.P.Shah Institute of Technology, Thane
Date:

Acknowledgement

We have great pleasure in presenting the report on **Sentiment-Augmented Stock Price Forecasting.** We take this opportunity to express our sincere thanks towards our guide **Ms.Ujwala Pagare** & Co-Guide **Ms.Aavani Nair** for providing the technical guidelines and suggestions regarding line of work. We would like to express our gratitude towards his constant encouragement, support and guidance through the development of project.

We thank **Ms. Anagha Aher** Head of Department for his encouragement during the progress meeting and for providing guidelines to write this report.

We express our gratitude towards BE project co-ordinators **Prof. Poonam Pangarkar**, for being encouraging throughout the course and for their guidance.

We also thank the entire staff of APSIT for their invaluable help rendered during the course of this work. WeguestoIT fortoencourto.

Sumit Shahu
(21107004)

Pravesh Yadav
(21107057)

Mohd Mustafa Shaikh
(21107045)

Ankit Purohit
(21107020)

Declaration

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, We have adequately cited and referenced the original sources. We also declare that We have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Signature)
Sumit Shahu(21107004)

(Signature)
Pravesh Yadav(21107057)

(Signature)
Mohd Mustafa Shaikh(21107045)

(Signature)
Ankit Purohit(21107020)

Date:

Abstract

This project presents a real-time stock price prediction framework that integrates deep learning models with sentiment analysis to enhance forecast accuracy during market fluctuations. The architecture uses Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) networks as base models to capture temporal dependencies in stock price data. An ensemble model then combines the outputs of LSTM and GRU to generate more robust predictions. To incorporate market sentiment, a separate RNN-based model analyzes the direction of news sentiment and influences the ensemble's final output. Sentiment analysis is performed using both BERT and VADER techniques, enabling the system to process live news feeds and social media updates. By merging sequential pattern recognition with real-time sentiment signals, the framework improves responsiveness to sudden market changes and supports more informed trading decisions.

Keywords: Deep Learning, LSTM, GRU, Ensemble Model, Sentiment Analysis, BERT, VADER, Stock Market Prediction, Time Series Forecasting, Financial Modeling, News Sentiment

Contents

1 Introduction **2**
- 1.1 Problem Statement . . . 3
- 1.2 Objectives . . . 4
- 1.3 Scope . . . 4

2 Literature Review **5**
- 2.1 Comparative Analysis of Recent Study . . . 5

3 Project Design **11**
- 3.1 Existing System . . . 11
  - 3.1.1 CNN-LSTM/GRU-CNN . . . 12
  - 3.1.2 Hybrid Statistical-ML Models . . . 13
- 3.2 Proposed System . . . 13
- 3.3 Critical Components of System Architecture . . . 15
  - 3.3.1 Deep Learning Models . . . 15
  - 3.4.1 UML Diagram . . . 16
  - 3.4.2 Activity Diagram . . . 18
  - 3.4.3 Use Case Diagram . . . 21
  - 3.4.4 Sequence Diagram . . . 22

4 Project Implementation **24**
- 4.1 Code Snippets . . . 24
- 4.2 Steps to access the System . . . 29
- 4.3 Timeline Sem VIII . . . 31
- 4.4 Project Management and Timeline . . . 31

5 Testing **34**

6 Results and Discussion **36**
- 6.1 System Implementation Framework . . . 36
  - 6.1.1 Data Acquisition Performance . . . 37
- 6.2 Temporal Pattern Analysis . . . 38
  - 6.2.1 Long-Term Trend Identification . . . 38
  - 6.2.2 Volatility Clustering . . . 38
- 6.3 Model Performance Evaluation . . . 39
  - 6.3.1 Individual Model Metrics . . . 39
  - 6.3.2 Ensemble Model Advantages . . . 39
- 6.4 Sentiment Integration Analysis . . . 40

6.4.1 News Impact Quantification 40
6.5 User Interaction Analysis 41
6.5.1 Decision Support Capabilities 41
6.5.2 Risk Management Features 41
6.6 Technical Challenges 41
6.6.1 Latency Bottlenecks 41
6.6.2 Data Quality Issues 42

7 Conclusions **43**

8 Future Directions and Emerging Opportunities **45**
8.1 Future Roadmap for Financial AI Systems 45
8.1.1 Next-Generation Model Architectures 46
8.1.2 Expanded Data Ecosystem 46
8.1.3 Real-Time Adaptive Systems 46
8.1.4 Ethical AI and Regulation 46
8.1.5 Decentralized Financial Intelligence 46
8.1.6 Specialized Financial Applications 46
8.1.7 Emerging Research Frontiers 46

Bibliography **47**

Appendix I: Python Libraries and Tools Used **49**

Appendix II: Dataset Information **50**

Publication **51**

List of Figures

3.1 System Architecture Diagram 14
3.2 High-Level System Overview 16
3.3 UML Component Diagram 18
3.4 System Workflow Diagram 20
3.5 User Interaction Diagram 22
3.6 System Sequence Flow 23

4.1 Gantt Chart 33

6.1 100-day vs 200-day moving average crossover patterns 38
6.2 GRU prediction vs actual prices during market shock 40

List of Tables

2.1 Comparative Analysis of Recent Studies 8

6.1 Comparative model performance 39

List of Abbreviations

LSTM	: Long Short-Term Memory
GRU	: Gated Recurrent Unit
RNN	: Recurrent Neural Network
ML	: Machine Learning
DL	: Deep Learning
NLP	: Natural Language Processing
BERT	: Bidirectional Encoder Representations from Transformers
VADER	: Valence Aware Dictionary and sEntiment Reasoner
MAE	: Mean Absolute Error
RMSE	: Root Mean Squared Error
LR	: Linear Regression

Chapter 1

Introduction

In today's fast-paced and interconnected financial ecosystem, accurate stock price forecasting remains a cornerstone for investment strategies, risk management, and automated trading systems. Traditional statistical models and linear regression techniques, although foundational, often falter under the pressure of nonlinear dependencies, high volatility, and the dynamic nature of real-time stock market data. In recent years, the fusion of financial modeling with deep learning and natural language processing (NLP) has ushered in a new era of intelligent forecasting tools. The stock market is not only influenced by historical trends and technical indicators but also by real-time sentiment data sourced from financial news articles, tweets, and social media trends. The motivation for developing a stock price forecasting system is to improve decision-making by leveraging advanced models to analyze complex market data and trends in real time. This enables investors to make informed choices, optimize investment strategies, and gain a competitive edge in an increasingly dynamic financial environment. Accurate forecasting not only enhances portfolio management but also supports risk mitigation and long-term financial planning. In a market where timely and precise insights can be the difference between profit and loss, the value of such a system becomes even more apparent. However, several challenges underline the need for such a system:

- **Market Volatility:** Modern stock markets are influenced by multiple volatile and interdependent factors—ranging from geopolitical events and macroeconomic policies to corporate performance and investor sentiment. These variables can shift rapidly, making the market highly unpredictable.
- **Increased Risk:** This unpredictability increases the level of risk associated with investments, complicating the decision-making process and often leading to suboptimal financial outcomes.
- **Limitations of Conventional Models:** Conventional forecasting models typically rely on historical trends and static assumptions, which are insufficient to capture the dynamic, nonlinear behavior of today's markets. They often struggle with adapting to sudden changes or anomalies, leading to inaccurate predictions.
- **Big Data Challenges:** The sheer volume and velocity of financial data generated in real time require models that are not only accurate but also efficient and scalable, capable of processing and interpreting vast datasets quickly and meaningfully.

- **Need for Intelligent Systems:** These challenges highlight the urgent need for intelligent forecasting systems that can learn, adapt, and evolve with the market—delivering insights that are both actionable and forward-looking.

1.1 Problem Statement

The problem of accurately forecasting stock prices continues to be one of the most intricate and unresolved challenges in the field of financial modeling and data science, primarily due to the highly volatile, dynamic, and non-linear behavior of the stock market. Stock prices are influenced by a wide array of unpredictable and interconnected factors such as historical pricing patterns, global economic indicators, breaking news events, investor psychology, and market sentiment expressed through various digital platforms including social media and financial news portals. Traditional forecasting methods and conventional machine learning models fall short in this context, as they are not equipped to efficiently process the massive volume of real-time data or capture the complex temporal dependencies required to make reliable predictions. These models often lack scalability, resulting in delays in prediction and limited accuracy, especially during periods of market turbulence. Moreover, they are ineffective in incorporating multiple heterogeneous data sources, such as numerical time-series data and unstructured textual information like sentiment extracted from financial news or tweets, which are increasingly recognized as critical indicators of market movement. The absence of a comprehensive system that can integrate these diverse inputs contributes to missed trading opportunities and suboptimal investment decisions.

- **Volatility & Prediction Challenges:** The stock market is highly volatile and difficult to predict in real-time due to rapidly changing market conditions, continuous fluctuations in pricing, unexpected geopolitical or economic news events, and the behavioral influence of investor sentiment.
- **Limitations of Existing Models:**
  - Inability to efficiently process real-time data.
  - Struggles to handle complex dependencies and multiple data sources.
  - Results in inaccurate predictions and delayed insights.
- **Accuracy:** Existing models often fail to detect intricate, nonlinear patterns in highly noisy and volatile datasets. This results in predictions that do not adapt well to abrupt market changes or anomalies.
- **Multiple Data Sources:** An effective prediction system must be capable of integrating multiple data streams—including numerical data (e.g., stock prices, trading volume, technical indicators) and unstructured textual data (e.g., news articles, social media sentiment). The lack of integration reduces prediction quality and ignores key contextual signals.
- **Scalability:** The volume of financial data is massive and constantly increasing. The system must be designed to scale effectively, handling high-frequency data while maintaining low response times and high accuracy.

1.2 Objectives

The primary objective of this project is to develop and evaluate a hybrid stock price forecasting system that effectively utilizes Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) models to capture both spatial and temporal patterns in financial market data. The system aims to enhance prediction accuracy and robustness by designing an ensemble stacking model that integrates LSTM, GRU, and Recurrent Neural Network (RNN) architectures. To further improve forecasting precision, the project incorporates sentiment analysis using advanced techniques such as BERT and VADER, enabling the model to factor in real-time news data and assess the impact of market sentiment on stock price movements. Additionally, the project compares the performance of deep learning approaches (LSTM, GRU, RNN) with traditional machine learning models like Random Forest and Gradient Boosting, evaluating them based on criteria such as accuracy, risk reduction, and applicability in real-world trading scenarios.

These are the project objectives:

- To develop and evaluate hybrid LSTM and GRU models for stock price forecasting, leveraging their ability to capture both spatial and temporal patterns in market data.
- To design and implement an ensemble stacking model that combines LSTM, GRU, and RNN architectures to enhance prediction accuracy and robustness across various market conditions.
- To integrate sentiment analysis using BERT and VADER techniques to incorporate real-time news data, assessing the impact of market sentiment on stock price predictions.
- To compare the performance of deep learning models (LSTM, GRU, RNN) and traditional methods (Random Forest, Gradient Boosting) in terms of accuracy, risk reduction, and real-world applicability.

1.3 Scope

A stock forecasting system predicts future stock prices using advanced machine learning models and real-time data from sources like historical prices, news, and social media. It helps investors and analysts make informed decisions, manage risks, and optimize strategies by identifying market trends and price fluctuations. The system is adaptable, scalable, and enhances financial decision-making across various market conditions. Some key future scopes include:

- **Adaptive Strategies:** Real-time forecasting allows traders to adjust strategies in response to sudden market changes or emerging trends.
- **Regulatory Compliance:** Helps firms monitor trading activities in real-time to ensure they follow regulations and quickly identify and report any suspicious behavior.
- **Investment Strategy Optimization:** Enables ongoing evaluation of investment strategies and market models to refine and enhance forecasting methods.

Chapter 2

Literature Review

Sentiment-augmented stock price forecasting integrates market sentiment derived from news articles, social media, and financial reports with traditional predictive models to enhance forecasting accuracy. This approach recognizes that investor sentiment significantly influences market behavior and can precede price movements. This literature review examines existing methodologies, datasets, and the effectiveness of various sentiment analysis techniques in improving stock price predictions.

2.1 Comparative Analysis of Recent Study

In recent years, numerous studies have been conducted to enhance the accuracy and reliability of stock price forecasting models. These research efforts explore various methodologies, including traditional statistical approaches, machine learning algorithms, and hybrid techniques that combine multiple models. This section presents a comparative analysis of selected recent studies, focusing on their objectives, methodologies, datasets, and performance outcomes. By evaluating the strengths and limitations of each approach, this analysis aims to identify prevailing trends, highlight innovative contributions, and uncover potential areas for further improvement in the field of stock market prediction.

In the paper[1] A stock market prediction approach using Transductive Long Short-Term Memory (TLSTM) and social media sentiment analysis, TLSTM enhances accuracy by focusing on recent data, improving short-term trend predictions. To address class imbalance in sentiment data, the Off-Policy Proximal Policy Optimization (PPO) algorithm adjusts rewards to prioritize underrepresented sentiments. This combination of stock data and sentiment analysis offers more accurate market predictions.

The proposed [2]MS-SSA-LSTM model combines multi-source data, including historical stock trading data and sentiment derived from stock forum comments, for improved predictive accuracy. The model utilizes the Sparrow Search Algorithm (SSA) to optimize the hyperparameters of the Long Short-Term Memory (LSTM) network, enhancing its ability to forecast stock prices. By incorporating sentiment analysis and optimizing LSTM parameters, the model shows improved performance, particularly in predicting short-term market movements, outperforming standard LSTM models by an average of 10.74%. The study emphasizes the importance of multi-source data in improving prediction accuracy, suggesting

that investor sentiment plays a critical role in stock price fluctuations.

In the research paper[3] A Deep Reinforcement Learning (DRL)-based Decision Support System for automated stock trading. It integrates both past and predicted future stock trends to enhance trading decisions. The model uses Gated Recurrent Units (GRUs) to forecast stock prices and incorporates this information into the state space of the Deep-Q Network (DQN) agent, enabling it to make more informed decisions on whether to buy, sell, or hold stocks. Tested on several stock markets, including Tesla, IBM, Amazon, and Chinese stocks, the model demonstrates improved profitability and efficiency in volatile market environments by optimizing trading strategies dynamically rather than relying on static rules.

The paper titled [4] "Decision Fusion for Stock Market Prediction: A Systematic Review" discusses how decision fusion techniques are applied in stock market prediction using machine learning and deep learning models. It highlights that predictions improve when multiple models (base learners) are combined, a method known as decision fusion or ensemble learning. The review examines trends over two decades, focusing on the characteristics of these base learners and the fusion methods applied. It explores different methods for classification and regression tasks and discusses future directions, including integrating new algorithms and sentiment analysis with decision fusion techniques.

The paper titled [5] "Stock Price Prediction Based on Morphological Similarity Clustering and Hierarchical Temporal Memory" introduces a new method for stock price prediction by clustering similar stocks using Morphological Similarity Distance (MSD) combined with k-means clustering. It then applies Hierarchical Temporal Memory (HTM), an online learning model, to learn patterns from these similar stocks, achieving better prediction accuracy compared to models that do not account for stock similarity. The results show that this method, called C-HTM, outperforms baseline models like LSTM and GRU, especially for short-term stock price predictions.

The paper titled [6] "A Deep Learning-Based Approach for Stock Price Prediction Using Bidirectional Gated Recurrent Unit and Bidirectional Long Short Term Memory Model" presents a comparative study between two deep learning models: Bidirectional Gated Recurrent Unit (BiGRU) and Bidirectional Long Short-Term Memory (BiLSTM). Both models are used to predict stock prices, leveraging forward and backward propagation to capture patterns in time-series data. The BiGRU model outperformed BiLSTM in terms of prediction accuracy, stability, and computational efficiency. The study used stock data from the NIFTY-50 index and demonstrated that BiGRU yielded better results across various metrics such as MSE, RMSE, and R2.

The paper [7] discusses the application of machine learning techniques for stock price prediction, emphasizing the limitations of traditional methods like fundamental and technical analysis. It highlights the use of Recurrent Neural Networks (RNN) and Long Short-Term Memory (LSTM) models due to their ability to handle sequential data, making them suitable for stock market prediction. By analyzing historical stock prices, these models predict future trends with improved accuracy compared to older methods. The paper also touches on the importance of reliable datasets and compares various models, concluding that RNN and LSTM are effective for stock trend prediction.

The paper[8] explores the use of sentiment analysis combined with machine learning to predict stock market movements. It highlights the difficulty in accurately predicting stock trends due to the market's inherent complexity. The authors analyze data from social media platforms like Twitter, extracting sentiment features such as polarity and subjectivity. By applying machine learning techniques, including model stacking, they attempt to improve prediction accuracy. The study examines the stock movements of six companies, achieving an accuracy of up to 60%. The research suggests that augmenting textual features with historical stock data enhances predictability but acknowledges the need for more refined methods.

The paper [9] by Sidra Mehtab and Jaydip Sen proposes a hybrid approach for predicting stock prices using machine learning and deep learning, particularly Convolutional Neural Networks (CNNs). It focuses on the NIFTY 50 index from India's National Stock Exchange over a four-year period (2015-2018) to predict stock movements for 2019. The authors use various classification and regression models to forecast stock values, emphasizing the use of CNNs due to their ability to capture complex patterns in multivariate time series data. They test different CNN configurations, including univariate and multivariate inputs, and find that CNN-based models significantly outperform traditional machine learning methods in prediction accuracy. The study highlights CNN's capability to improve stock price forecasting, especially for medium-term investors.

The paper titled [10] "A Prediction Approach for Stock Market Volatility Based on Time Series Data" explores stock market forecasting through time series analysis, focusing on the Indian stock market. It employs the ARIMA (Auto-Regressive Integrated Moving Average) model to predict stock trends for Nifty and Sensex indices using historical data from 2012 to 2016. The study emphasizes the importance of financial forecasting for decision-making and risk management, highlighting how ARIMA helps transform non-stationary data into a stationary format for accurate predictions. Validation methods, such as the Augmented Dickey-Fuller and L-Jung Box tests, confirm the reliability of the predictions, with the results showing a mean error margin of around 5%. This forecasting model is proposed as useful not only in finance but also in various other domains like health and education.

The paper [11] "Predicting Prices of Stock Market using Gated Recurrent Units (GRUs) Neural Networks" explores the application of GRUs, a type of recurrent neural network, for predicting stock market prices. The authors propose modifications to the GRU's internal structure to overcome challenges such as the vanishing gradient problem and local minima, aiming to improve efficiency and reduce computation time. Using real-time stock data from Yahoo Finance, the model is trained and evaluated through mini-batch gradient descent, with accuracy measured via Root Mean Square Error (RMSE). Results show that GRUs effectively learn patterns from historical data and outperform other recurrent networks like LSTMs in certain aspects. The paper concludes with future prospects of developing automated trading agents based on the proposed system.

The paper [12] proposes a deep learning model to predict short-term stock trends using a two-stream Gated Recurrent Unit (TGRU) network. It integrates financial news articles and historical stock prices, leveraging both sentiment analysis and Stock2Vec, a sentiment

embedding model trained on financial data. The study conducts two experiments: one using SP 500 index data and news from Reuters and Bloomberg, and another using the VN-index and VietStock news. The results demonstrate that the TGRU outperforms other models, achieving higher accuracy and effectively predicting stock price directions in both datasets. The model also shows robustness in real-world trading simulations.

Table 2.1: Comparative Analysis of Recent Studies

Sr. No	Title	Author(s)	Year	Methodology	Drawback
1	Stock Market Prediction With Transductive Long Short-Term Memory and Social Media Sentiment Analysis [1]	Ali Peivandizadeh, Sima Hatami, Amirhossein Nakhjavani, Lida Khoshsima, Mohammad Reza Chalak Qazani, Muhammad Haleem, Roohallah Alizadehsani	2024	Combines TLSTM (Transductive LSTM) for stock price prediction with social media sentiment analysis. Uses Off-Policy Proximal Policy Optimization (PPO) to handle class imbalances.	Over-reliance on sentiment from social media.Complex architecture requires significant computational resources. Data imbalances in sentiment analysis can still affect the model's performance.
2	A Stock Price Prediction Model Based on Investor Sentiment and Optimized Deep Learning [2]	Guangyu Mu, Nan Gao, Yuhan Wang, Li Dai	2023	MS-SSA-LSTM model integrates multi-source data, such as: Investor sentiment from forums , Stock trading data. Optimizes LSTM (Long Short-Term Memory) using the Sparrow Search Algorithm. Uses a sentiment dictionary to improve prediction accuracy.	model heavily depends on sentiment data Sentiment categorization is limited to positive and negative emotions. Lacks integration of more complex emotional aspects such as fear, anger, etc. .
3	A Deep Reinforcement Learning-Based Decision Support System for Automated Stock Market Trading [3]	Yasmeen Ansari, Sadaf Yasmin, Sheneela Naz, Hira Zaffar, Zeeshan Ali, Jihoon Moon, Seungmin Rho	2022	Deep Reinforcement Learning (DRL) framework using Deep-Q Networks with Gated Recurrent Unit (GRU) forecasting. The model uses both past and future stock trends to make trading decisions.	Model performance is volatile and may vary with different training attempts. Lacks robustness in highly volatile markets, and future price predictions are sometimes inaccurate, affecting trading decisions.
4	Decision Fusion for Stock Market Prediction: A Systematic Review [4]	Cheng Zhang, Nilam N. A. Sjarif, Roslina B. Ibrahim	2022	Decision fusion: Combines forecasts from multiple models using base learners and fusion methods	Few studies use decision fusion.Lack of diversity in ensemble models.Minimal integration of sentiment analysis.

Sr. No	Title	Author(s)	Year	Methodology	Drawback
5	Stock Price Prediction Based on Morphological Similarity Clustering and Hierarchical Temporal Memory [5]	Xingqi Wang, Kai Yang, Tailian Liu	2022	K-means clustering with Morphological Similarity Distance (MSD) to group stocks, followed by Hierarchical Temporal Memory (HTM) for prediction	. The method may not generalize well to long-term predictions.Multivariate input had minimal benefit for HTM. Potential underfitting or overfitting of baseline models due to lack of parameter tuning.
6	A Deep Learning-Based Approach for Stock Price Prediction Using Bidirectional Gated Recurrent Unit and Bidirectional Long Short Term Memory Model [6]	Md. Ebtidaul Karim, Sabrina Ahmed	2021	BiGRU (Bidirectional Gated Recurrent Unit) with activation layer andBiLSTM (Bidirectional Long Short Term Memory).	Increased complexity with more hidden layers. Trainable parameters grow with additional layers, which can increase computation cost. Limited to comparison between BiGRU and BiLSTM models.
7	Literature Survey on Stock Price Prediction Using Machine Learning [7]	Anusha J Adhikar, Apeksha K Jadhav, Charitha G, Karishma KH, Mrs. Supriya HS	2020	Machine Learning: Recurrent Neural Networks (RNN), Long Short Term Memory (LSTM). Data collected from Yahoo Finance.	Python libraries used were not optimal for training speed. Need for optimization of neural network parameters.
8	Augmented Textual Features-Based Stock Market Prediction [8]	Salah Bouktif, Ali Fiaz, Mamoun Awad	2020	Sentiment analysis using tweets and stock data combined with machine learning techniques such as SVM, Naive Bayes, ANN, and XGBoost with model stacking.	Limited by the text features extraction approach, Detailed analysis of sentiments is needed to improve prediction accuracy

Sr. No	Title	Author(s)	Year	Methodology	Drawback
9	Stock Price Prediction Using Convolutional Neural Networks on a Multivariate Time-series [9]	Sidra Mehtab, Jaydip Sen	2020	Machine Learning: Logistic Regression, KNN, Decision Tree, Bagging, Boosting, Random Forest, ANN, SVM. Deep Learning: CNN with three approaches: univariate, multivariate, and multi-model. Data from NIFTY 50 index (2015- 2019).	Difficulty in predicting rapid stock price changes. Performance varies with data size and model settings. Higher prediction errors (RMSE) in some CNN configurations.
10	A Prediction Approach for Stock Market Volatility Based on Time Series Data [10]	Sheikh Mohammad Idrees, M. Afshar Alam, Parul Agarwal	2019	ARIMA (Auto-Regressive Integrated Moving Average) model for time series data prediction	Limited to univariate data, assuming no external factors like social or economic events.
11	Predicting Prices of Stock Market using Gated Recurrent Units (GRUs) Neural Networks [11]	Mohammad Obaidur Rahman, Md. Sabir Hossain, Ta-Seen Junaid, Md. Shafiul Alam Forhad, Muhammad Kamal Hossen	2019	Gated Recurrent Units (GRUs) for stock price prediction Mini-batch gradient descent Real-time dataset from Yahoo Finance	Local minima problem and time complexity of stochastic gradient descent were reduced but still may arise.
12	Deep Learning Approach for Short-Term Stock Trends Prediction Based on Two-Stream Gated Recurrent Unit Network [12]	Dang Lien Minh, Abolghasem Sadeghi-Niaraki, Huynh Duc Huy, Kyungbok Min, Hyeonjoon Moon	2019	Two-Stream Gated Recurrent Unit (TGRU) Network, Sentiment analysis with Stock2Vec, financial news data, and technical indicators.	Complexity of TGRU model, requiring long training times and computational resourcesLimited to daily stock movements rather than intra-day trading.

Chapter 3

Project Design

In the design of our sentiment-infused stock price prediction system, meticulous attention was given to capturing both the temporal patterns in historical stock data and the dynamic sentiment-driven fluctuations that are frequently triggered by real-world events, news, and social media activity. The design philosophy revolves around developing a robust, scalable, and intelligent framework capable of real-time analytics and seamless integration of diverse data streams. By combining the powerful sequential modeling capabilities of Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) architectures with the expressive strengths of sentiment analysis models, the project aims to bridge the gap between technical analysis and behavioral finance. A core challenge addressed during design was to enable simultaneous processing of structured time-series data and unstructured sentiment data while ensuring efficient data flow, modularity, and model interpretability. The system employs an ensemble learning strategy to merge multiple predictive signals—derived from historical trends and real-time news headlines—into a unified output that strengthens prediction accuracy and minimizes market noise. A real-time data ingestion pipeline ensures timely updates, while backend integration with financial APIs and custom-built scrapers makes the system reactive to fast-changing market dynamics. Moreover, user-centric considerations were integrated into the interface design, allowing investors and traders to interact with the system intuitively, view predictive insights through graphs and analytics, and make informed decisions. Each design component, from data acquisition to sentiment scoring and model training, has been tailored for extensibility and future enhancements, thereby laying the foundation for a smart, adaptive, and insightful stock forecasting tool tailored for today's fast-paced financial ecosystem.

3.1 Existing System

Over the years, numerous approaches have been developed to forecast stock prices using traditional statistical models and machine learning techniques. These include methods like linear regression, ARIMA models, support vector machines, and decision trees. However, these conventional approaches often fall short when faced with the highly dynamic and non-linear nature of stock market data, especially in the context of real-time forecasting. They are generally limited in their ability to handle complex temporal dependencies, adapt to sudden market fluctuations, or process large-scale, heterogeneous data streams in real-time. As the financial domain has rapidly evolved with increasing data granularity and real-time requirements, more advanced systems have emerged that leverage deep learning architec-

tures. Among these, hybrid models that combine Convolutional Neural Networks (CNNs) with Recurrent Neural Networks (RNNs), particularly Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU), have shown significant promise. These models are capable of learning both spatial and temporal features, enabling more accurate and context-aware predictions. In such architectures, CNN layers are typically used for extracting local patterns and noise reduction from time-series data, while LSTM or GRU layers are responsible for learning long-term dependencies and sequential trends. Additionally, ensemble learning techniques have been integrated to combine the predictive power of multiple base models, improving overall robustness and generalization. Some systems also incorporate hybrid statistical and machine learning models, such as ARIMA-ANN and Wavelet-ANN, to blend the strengths of both paradigms. Despite these advancements, current systems still face several limitations—particularly in integrating real-time sentiment analysis, adapting quickly to volatile market changes, and maintaining low-latency, high-frequency data processing. These gaps in the existing systems highlight the need for a more comprehensive and responsive forecasting model that can seamlessly combine time-series analysis, deep learning, and sentiment-based signals.

The existing systems for stock price forecasting primarily rely on traditional statistical models and basic machine learning techniques, which often fall short in capturing the complex, nonlinear, and dynamic nature of financial markets. These conventional approaches struggle with handling real-time data, adapting to sudden market changes, and integrating diverse data sources such as technical indicators and market sentiment. In recent advancements, hybrid models combining Convolutional Neural Networks (CNNs) with Recurrent Neural Networks (RNNs), including LSTM and GRU architectures, have shown promise in improving prediction accuracy. These models utilize the strengths of CNNs for spatial feature extraction and RNNs for learning temporal dependencies in stock data. Additionally, ensemble models and hybrid statistical-ML frameworks have been employed to mitigate the limitations of individual models by leveraging their combined predictive capabilities. Despite these improvements, existing systems still face challenges in effectively integrating real-time sentiment data and delivering timely, accurate forecasts across varying market conditions.

3.1.1 CNN-LSTM/GRU-CNN

These models combine convolutional layers (for spatial feature extraction) with recurrent layers (LSTM/GRU for temporal dependencies). For example:

- **Forecasting Stock Market Indices Using Hybrid Models:** Proposes CNN-LSTM and GRU-CNN models where CNNs preprocess time-series data to reduce noise before feeding into LSTM/GRU layers. This improves feature learning for stock indices like DAX and S&P500.
- **Stock Price Prediction Using Multivariate CNNs:** Uses channel-wise inputs (Open, High, Low, Close) to predict NIFTY 50 trends, demonstrating CNN's strength in handling multi-dimensional data.
- **Multi-Headed CNNs:** Employs separate CNN sub-models for each input variable (e.g., Open, High) and merges their outputs, enhancing prediction robustness for NIFTY 50.

3.1.2 Hybrid Statistical-ML Models

- **ARIMA-ANN/Wavelet-ANN:** Hybrid models integrate statistical methods with neural networks to handle non-linearities and noise in stock data.

3.2 Proposed System

To address the limitations observed in existing stock forecasting systems, the proposed solution introduces a hybrid, ensemble-based architecture that combines both historical financial data and sentiment analysis to produce more accurate, context-aware stock price predictions. This system leverages the strengths of advanced deep learning models—specifically Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) networks—which are well-suited for capturing the complex temporal dependencies in time-series data such as stock prices. LSTM networks, with their internal memory gates, are capable of retaining long-term patterns and filtering out irrelevant information, making them particularly effective in modeling the sequential behavior of stock movements. Similarly, GRUs offer comparable capabilities with a more lightweight architecture, leading to reduced training times and computational overhead.

In the proposed system, both LSTM and GRU models are trained on sequences of historical stock data, including attributes like open, high, low, close, and volume, enabling the system to identify patterns and forecast short- to mid-term price movements. To enhance the robustness and reliability of predictions, the outputs of these models are then combined using an ensemble learning approach. In this approach, a meta-model—typically a linear regression model—is trained on the stacked predictions from the base models (LSTM and GRU). This ensemble technique reduces the variance and improves generalization, especially under volatile market conditions.

Additionally, the system incorporates sentiment analysis as a supplementary signal to influence price predictions. This is achieved by deploying a Recurrent Neural Network (RNN)-based sentiment model, integrated with the BERT model, to analyze the sentiment polarity of financial news articles, company reports, and social media content fetched using NewsAPI. The sentiment scores extracted from this analysis are then incorporated into the ensemble framework as additional features, enriching the forecasting context with real-world investor sentiment and market psychology.

The architecture is designed to be scalable and adaptive, supporting continuous updates through data ingestion techniques such as the yfinance API for stock data and NewsAPI for news data. This allows the system to operate effectively with up-to-date financial data and sentiment signals. Preprocessing modules are employed to handle normalization, transformation, and feature engineering, ensuring that the data fed into the models is of high quality. The final output of the system is a forecast of the next trading day's closing price, with the predictions adjusted based on the sentiment-driven influence.

This holistic approach bridges the gap between quantitative historical data and qualitative market sentiment, providing a scalable and adaptive forecasting mechanism capable of supporting decision-making in data-intensive financial markets.

The web application for stock forecasting utilizes a combination of advanced machine learning techniques. An ensemble model (Figure 3.1) will then integrate the outputs from the

LSTM, GRU, and RNN sentiment analysis to produce a more robust and comprehensive stock forecast, offering investors and traders a well-rounded prediction tool that accounts for both quantitative data and qualitative market sentiment. This integration aims to improve prediction accuracy, helping users make informed trading decisions in real time.

- LSTM/GRU models for historical stock data analysis
- RNN for real-time news sentiment analysis
- Ensemble model integrating both data streams

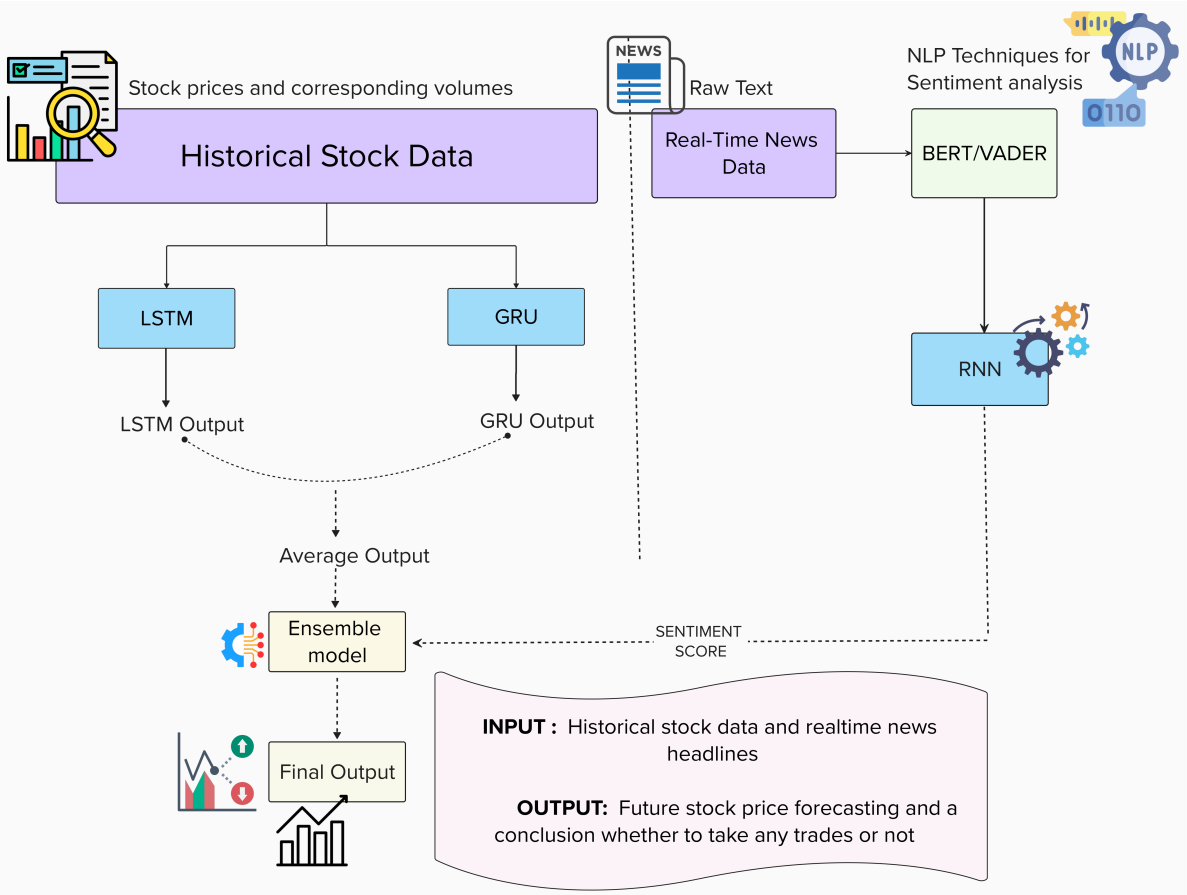

Figure 3.1: System Architecture Diagram

Recurrent Neural Networks (RNNs) are designed to process sequential data by maintaining a hidden state that captures information from previous inputs, making them suitable for tasks like time series forecasting and language modeling. However, RNNs often struggle with long-range dependencies due to issues such as vanishing gradients. Long Short-Term Memory (LSTM) networks address this limitation by introducing memory cells and gates (input, output, and forget gates) that regulate the flow of information, allowing them to retain long-term dependencies more effectively. This makes LSTMs particularly useful in applications like natural language processing and speech recognition. Gated Recurrent Units (GRUs) offer a simpler alternative to LSTMs by combining the input and forget gates into a single update gate, resulting in fewer parameters while still performing well on similar tasks.

Overall, while RNNs serve as a foundational architecture for sequential data, LSTMs and GRUs are advanced models that excel in capturing long-term dependencies.

3.3 Critical Components of System Architecture

The critical components of the proposed system architecture are designed to effectively address the challenges of real-time stock price forecasting by leveraging advanced deep learning techniques. At the core of the architecture are specialized models such as Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) networks, which are well-suited for capturing long-term dependencies and sequential patterns in time-series financial data. These models work in conjunction with a Recurrent Neural Network (RNN) that performs sentiment analysis on real-time news data, providing valuable contextual insights that influence market behavior. Each component plays a crucial role in the overall system, from data preprocessing and feature extraction to model training and prediction. The integration of these models through an ensemble learning approach ensures improved robustness, accuracy, and adaptability, allowing the system to respond effectively to rapidly changing market conditions.

3.3.1 Deep Learning Models

This section outlines the core deep learning models used in the system for time-series stock price forecasting. The architecture primarily leverages Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) networks due to their ability to capture sequential dependencies and long-term patterns in financial data. These models are well-suited for handling the temporal nature of stock market trends and are optimized using dropout layers and appropriate loss functions to prevent overfitting. Additionally, a simple Recurrent Neural Network (RNN) is employed for processing sequential text data during sentiment analysis. The combined use of these models ensures accurate, robust, and context-aware predictions, forming the foundation of the system's forecasting engine.

System diagrams play a crucial role in visualizing the structural and operational framework of the proposed sentiment-augmented stock price forecasting system. These diagrams encapsulate the dynamic interaction between various components and illustrate how data flows through the system, from initial collection to final prediction. As depicted in Figure 3.2, the system architecture is designed with three primary parallel data pipelines: the first processes historical stock market data using Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) models to capture temporal trends and long-range dependencies; the second focuses on real-time textual data from financial news sources and social media platforms, utilizing a Recurrent Neural Network (RNN) integrated with NLP models such as FinBERT or VADER to extract and quantify sentiment; and the third pipeline includes data preprocessing modules responsible for scaling, cleaning, and formatting both numerical and textual inputs. All three pipelines converge at the ensemble integration layer, where outputs from the LSTM and GRU models are stacked and combined using a meta-model—typically a linear regression model—alongside sentiment scores to produce final predictions.

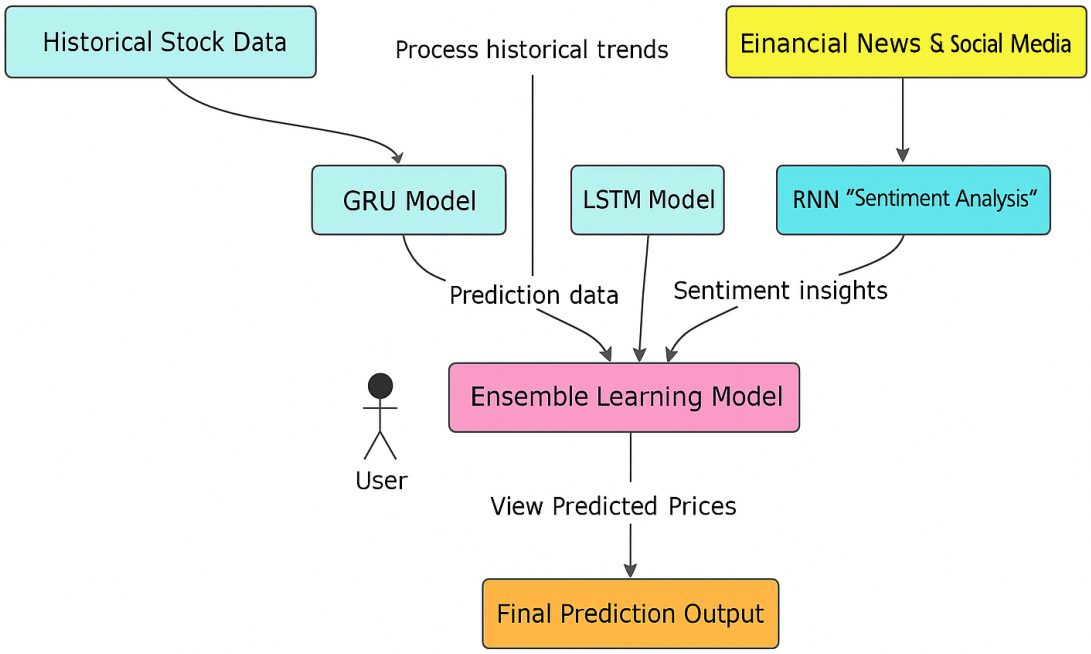

Figure 3.2: High-Level System Overview

This architectural flow is carefully orchestrated to maintain low latency and high accuracy, particularly in high-frequency trading scenarios. Supporting this core architecture, additional UML component diagrams, activity diagrams, and use case diagrams are employed to break down functional responsibilities, identify user-system interactions, and visualize sequential workflows. For example, the UML diagram outlines the collaboration between subsystems like the API handler, model trainer, sentiment extractor, and prediction engine, while the activity diagram traces the lifecycle of a prediction request from user input to graphical output. The use case diagram highlights major features such as forecasting, viewing sentiment trends, and setting prediction thresholds, emphasizing the interaction between different stakeholders like investors and analysts. Meanwhile, the sequence diagram delineates the time-ordered exchange of data between components, making it easier to understand processing delays or potential bottlenecks. Together, these system diagrams not only clarify the internal mechanics of the forecasting system but also ensure robust design, maintainability, and effective team collaboration.

Key components shown in Figure 3.2 include:

- Three parallel data processing pipelines
- Ensemble learning integration point
- Real-time data feeds and user interface

3.4.1 UML Diagram

The UML (Unified Modeling Language) diagram plays a vital role in illustrating the structural and functional architecture of the sentiment-augmented stock price forecasting system.

It provides a comprehensive visualization of the system's components, their relationships, and the interactions between internal modules and external entities such as users or data sources. This diagram helps both developers and stakeholders understand how the system is logically organized, ensuring clarity in design and efficient communication across the development team. At the core of the UML diagram is the Forecast Engine, which acts as the central module orchestrating interactions between major functional blocks such as the Data Ingestion Module, Preprocessing Unit, Model Handler, Ensemble Layer, and User Interface. The Data Ingestion Module interfaces with external APIs and web sources to collect live stock market data, historical prices, technical indicators, and financial news headlines. The Preprocessing Unit receives this data and performs tasks like normalization, feature extraction, noise filtering, and transformation of textual inputs into numerical sentiment scores using techniques such as BERT or VADER.

Once the data is cleaned and formatted, it is sent to the Model Handler, which encapsulates three deep learning models: LSTM, GRU, and an RNN for sentiment analysis. These models operate in parallel to capture sequential patterns, long-term dependencies, and emotional cues from the market. Their outputs are then directed to the Ensemble Module, where a stacking mechanism is used to combine the predictions and generate a more accurate, robust final output. This prediction is forwarded to the User Interface, allowing users—primarily investors, analysts, or researchers—to interact with the system through visualization tools, customizable settings, and sentiment-aware forecasting insights. Supporting components like the Model Trainer and Scheduler are also shown in the UML diagram, responsible for periodic model retraining, continuous learning from updated data, and maintaining synchronization across all pipelines.

The UML diagram clearly highlights dependencies and communications between modules through directional relationships and connectors, making it easier to identify how changes in one module might affect others. By modeling these interactions in a structured and visual manner, the diagram helps ensure scalability, modularity, and maintainability. Overall, the UML diagram not only guides the system's implementation but also supports its long-term evolution, serving as a blueprint for both current development and future enhancements.

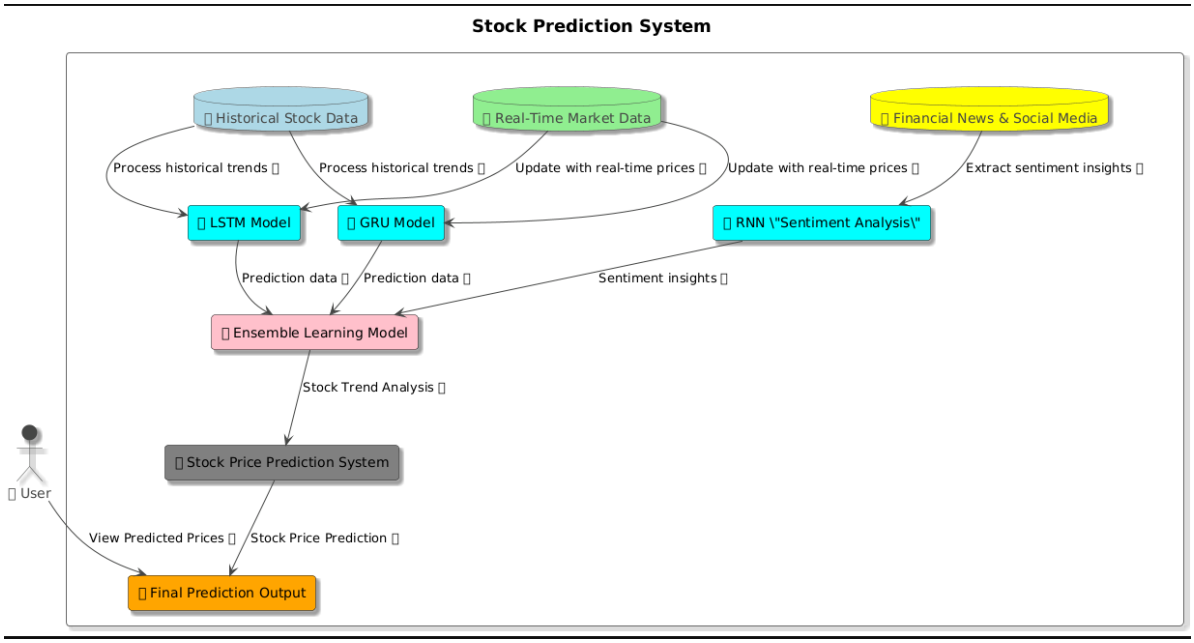

Figure 3.3: UML Component Diagram

Figure 3.3 illustrates:

- Actor-system interactions
- Data source integrations
- Model collaboration patterns

3.4.2 Activity Diagram

The Activity Diagram offers a dynamic and process-oriented view of the sentiment-augmented stock price forecasting system by illustrating the flow of control between various activities involved in processing and predicting stock prices. It captures the operational logic and the sequence of steps that occur from the moment a user initiates a prediction request to the point where the final forecast is generated and displayed. This diagram begins with the user accessing the system through the graphical interface and selecting a specific stock or index for analysis. Once the input is received, the system transitions to the data acquisition stage, where two types of data are fetched in parallel—historical stock market data from financial APIs, and real-time textual data from news sources and social media platforms. The activity flow branches here, depicting the parallelism in the system's architecture.

On one side, the historical stock data is routed to the preprocessing module, where it is cleaned, normalized, and structured into time-series sequences suitable for deep learning models. Simultaneously, the textual data passes through natural language processing pipelines for tokenization, sentiment extraction, and transformation into sentiment scores using models like FinBERT or VADER. These sentiment scores are essential in identifying the market's emotional state, such as optimism, fear, or uncertainty. Once both data streams are preprocessed, they are forwarded to the modeling stage. Here, the historical data is fed into LSTM and GRU models, which learn temporal patterns, while the sentiment data is

processed through an RNN to extract behavioral insights. The next activity combines all the outputs through the ensemble learning mechanism, typically using a linear regression meta-model to stack the predictions from LSTM and GRU, enriched by the sentiment features.

Following this, the ensemble output undergoes inverse transformation to convert scaled predictions back to actual price values, which are then sent to the presentation module. The system concludes by generating visual outputs such as charts, forecast summaries, and sentiment overlays for the user. The activity diagram also includes decision nodes—such as whether sufficient data is available, whether the sentiment analysis was successful, or if the model needs to be retrained—ensuring that fallback procedures and alternate flows are clearly represented. Feedback loops are embedded to allow continuous learning, where the model updates itself with newly available data over time. This holistic view not only helps in visualizing system workflows but also supports optimization, debugging, and effective communication between development teams, testers, and non-technical stakeholders.

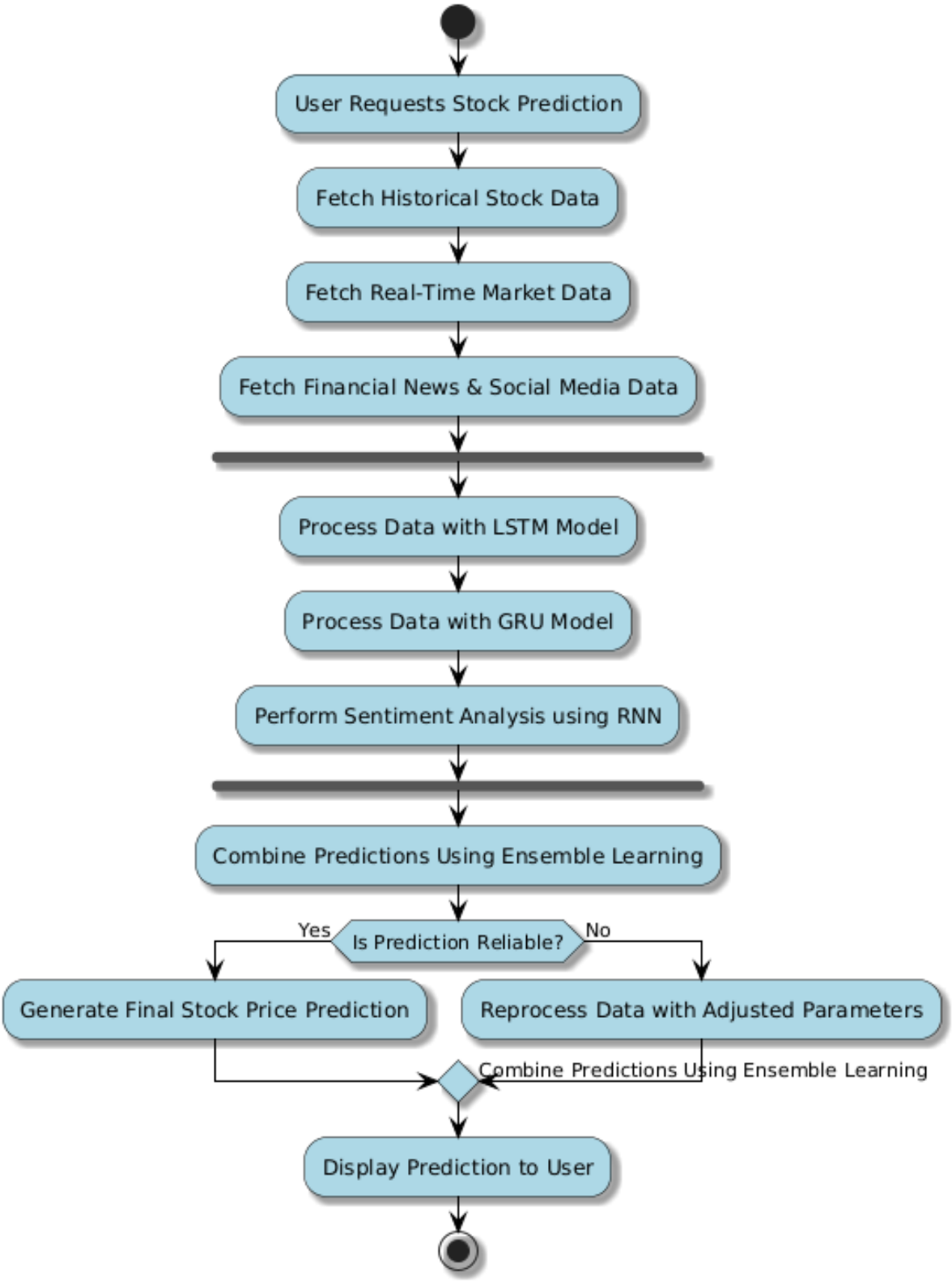

Figure 3.4: System Workflow Diagram

- Parallel processing branches
- Ensemble reliability checks

- Feedback loops for parameter adjustment

3.4.3 Use Case Diagram

A use case diagram for stock price forecasting illustrates the interactions between various users and the forecasting system. Key actors include the Investor, who seeks to make informed investment decisions; the Analyst, who analyzes market trends; and the Data Source, which provides historical stock prices and relevant market data. The primary use cases encompass several functionalities: users can input historical data into the system, which allows for analyzing market trends to inform predictions. The core function, generate forecast, employs algorithms to predict future stock prices based on historical data. Users can then visualize the forecast through graphs or charts, facilitating better understanding and interpretation. Additionally, there's an option to evaluate forecast accuracy by comparing predicted prices with actual outcomes, enabling users to refine their approaches. Finally, the Investor leverages these forecasts to make investment decisions regarding buying or selling stocks. This diagram effectively captures the essential workflows and relationships involved in the stock price forecasting process. The use case diagram for the stock forecasting system outlines key interactions between the user and the system, emphasizing five main use cases:

- **View Stock Price Forecast**: Users can access predicted stock prices to make informed investment decisions.
- **Access Real-Time News**: Users can read the latest news articles relevant to the stock market, helping them stay updated on events that may affect stock prices.
- **Generate Sentiment Analysis**: This feature allows users to analyze news sentiment, providing insights into market trends based on the tone of news articles.
- **Adjust Forecast Parameters**: Users can modify parameters such as forecasting models and timeframes, enabling customization to suit their investment strategies.
- **Receive Alerts and Notifications**: Users can set up alerts for significant price changes or sentiment shifts, ensuring they stay informed of critical developments.

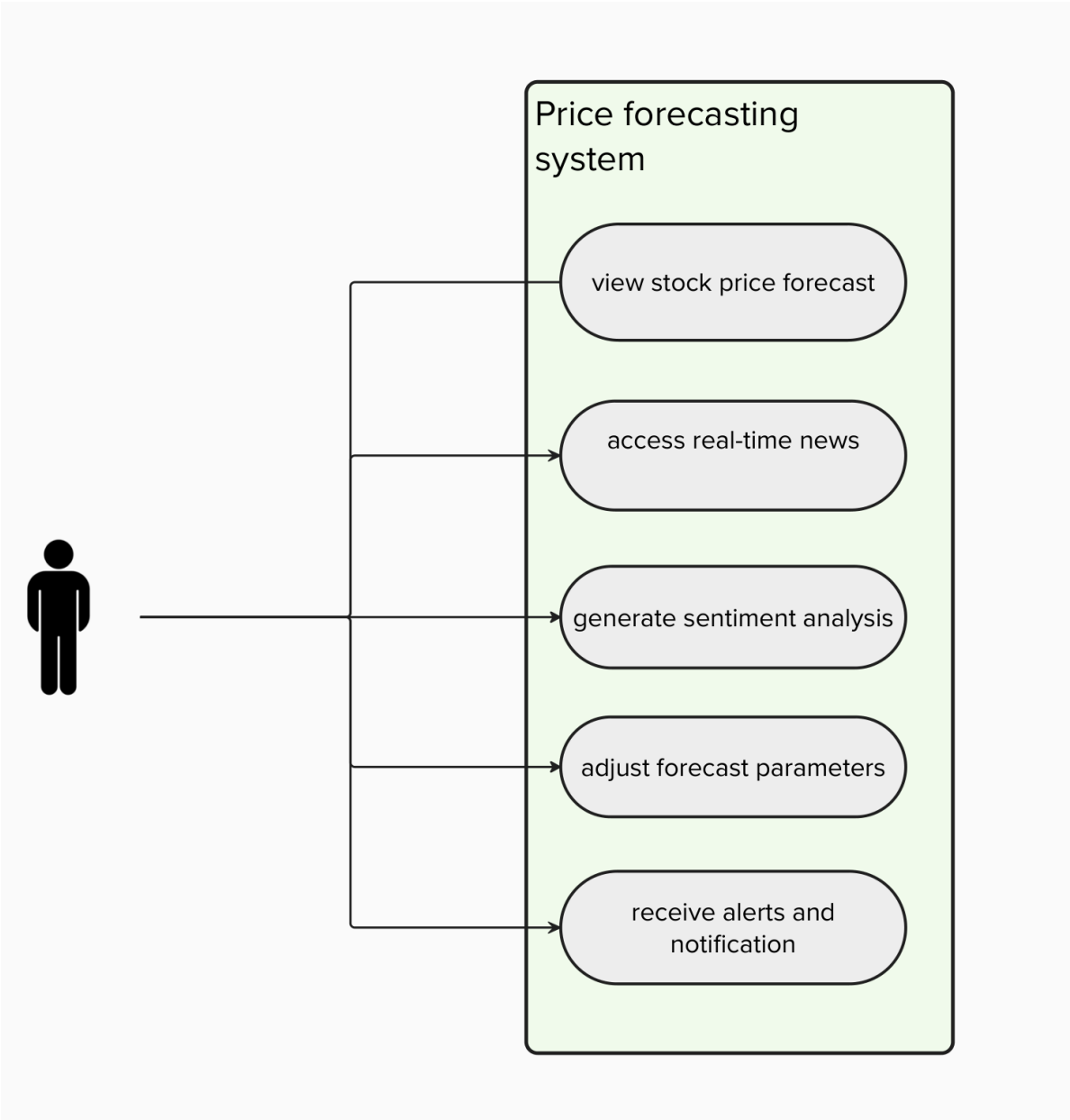

Figure 3.5: User Interaction Diagram

3.4.4 Sequence Diagram

The Sequence Diagram provides a time-based, interactive visualization of how the different components of the sentiment-augmented stock price forecasting system communicate and exchange data during the execution of a prediction request. It focuses on the chronological order of message passing between entities such as the User Interface, Data Acquisition Module, Preprocessing Unit, Model Handler (LSTM, GRU, RNN), Ensemble Engine, and the Visualization Layer. The interaction begins when the user initiates a stock prediction by entering a company name or selecting a stock from a predefined list via the web-based interface. This request is first sent to the Controller or API Gateway, which acts as the central dispatcher for incoming commands. The Controller simultaneously sends asynchronous requests to the Data Acquisition Module to fetch both historical stock price data and real-

time news content from financial APIs and web-scraped sources.
Once the data is retrieved, it is sequentially passed to the Preprocessing Unit, where the stock data is normalized using techniques like Min-Max scaling and reshaped into 3D arrays for compatibility with time-series models. Meanwhile, the textual data undergoes natural language processing (NLP), involving tokenization, stop-word removal, sentiment analysis, and conversion into numerical sentiment scores using models like VADER or BERT. The sequence continues as the preprocessed time-series data is forwarded to two separate deep learning pipelines: one for the LSTM model and another for the GRU model. Simultaneously, the sentiment data is input into a Recurrent Neural Network (RNN) model trained specifically for real-time sentiment classification. Each of these models processes the inputs and returns predictions to the Ensemble Engine.
The Ensemble Engine then receives these model-specific outputs, aligns them into a unified format, and applies a stacking method using a meta-model—often a simple linear regression—to combine the results into a final, more accurate prediction. This final forecast is passed back to the Controller, which then relays it to the Visualization Layer. Here, the results are formatted into user-friendly visual elements such as forecast charts, trend lines, and sentiment overlays, allowing users to interpret the outcomes intuitively. Throughout the entire sequence, time-ordered arrows and activation boxes in the diagram show when each component is active and which processes are synchronous or asynchronous. This makes it easy to identify dependencies, possible bottlenecks, and delays in processing. The Sequence Diagram is particularly valuable for understanding the system's real-time behavior, ensuring efficient module integration, and facilitating scalability. It also supports troubleshooting and optimization by revealing the precise flow of information during every step of the forecasting pipeline.

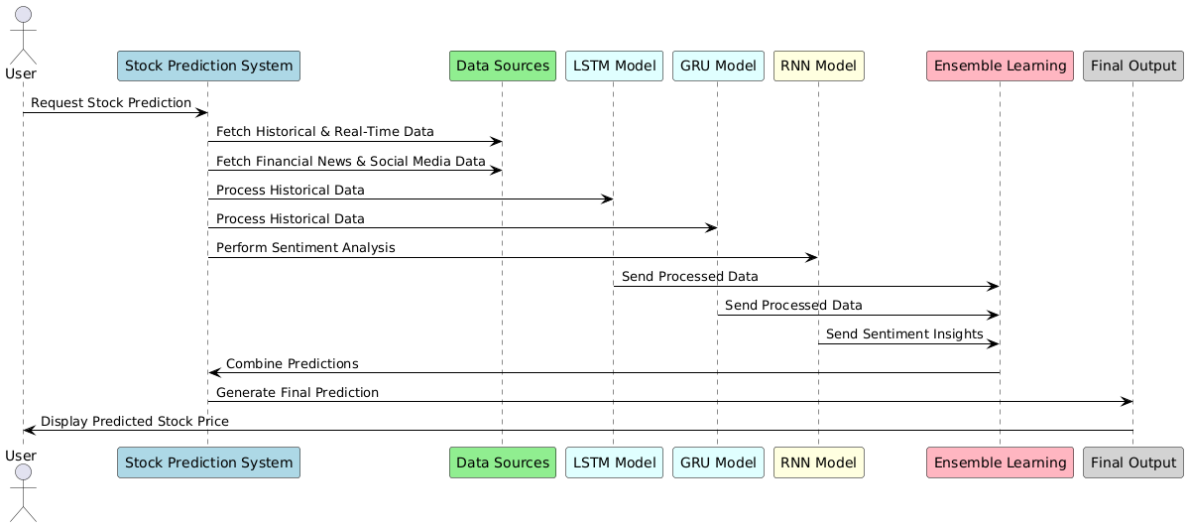

Figure 3.6: System Sequence Flow

The sequence diagram in Figure 3.6 demonstrates:

- Data collection sequence
- Parallel model processing
- Ensemble aggregation flow

Chapter 4

Project Implementation

The implementation phase of this project focuses on translating the conceptual design and system architecture into a functional prototype capable of performing real-time stock price forecasting enriched with sentiment analysis. This chapter outlines the practical steps taken to build the system, including data collection, preprocessing, model construction, training, testing, and integration of ensemble predictions. The system is implemented using Python due to its extensive libraries for data science, deep learning, and natural language processing. Key libraries include TensorFlow and Keras for model development, Pandas and NumPy for data manipulation, Scikit-learn for ensemble techniques, and NLTK/VADER or Transformers for sentiment extraction. The implementation follows a modular approach, where each component—data ingestion, preprocessing, deep learning model, sentiment analysis, and prediction engine—is built and tested independently before being integrated into the full pipeline. In the following sections, code snippets are presented to illustrate how critical parts of the system have been developed and executed.

4.1 Code Snippets

The following code snippets represent the core implementation logic of the Sentiment-Augmented Stock Price Forecasting system. These snippets demonstrate the integration of deep learning models—specifically Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) networks—with real-time sentiment analysis for enhanced prediction accuracy. The code is structured using modular design principles, including data preprocessing, model training, prediction stacking, and real-time forecasting. Each model is trained using historical stock price data with scaled features, and predictions are then combined using a meta-learning approach through Linear Regression. Additionally, sentiment analysis results—generated from news headlines using FinBERT—are integrated as additional input features. The snippets also showcase the development of a forecasting interface using Python, Flask, and supporting libraries, enabling users to interact with the models via a web-based application. Together, these code segments form the technical foundation of the end-to-end forecasting pipeline.

LSTM and GRU Networks

Both models are structured with two recurrent layers consisting of 128 and 64 units, respectively. Dropout layers with a rate of 0.2 are inserted after each recurrent layer to mitigate

overfitting.

The final layers include:

- A dense layer with 32 units
- A single-output dense layer for regression

The input shape to the models is (time_steps, 4), indicating sequences of 60 days (time_steps = 60) with 4 features per timestep (e.g., Open, High, Low, Close).

The models are compiled using the Adam optimizer and the Mean Squared Error (MSE) loss function, which is standard for regression tasks.

Training is conducted over 50 epochs with a batch size of 32, using the scaled training data ($X_{\text{train}}, y_{\text{train}}$).

Model Persistence

Trained models (Figure 4.1) are saved in `.h5` format for reuse:

- `lstm_model.h5`
- `gru_model.h5`

```
lstm_model = Sequential([
    LSTM(128, return_sequences=True, input_shape=(time_steps, 4)),
    Dropout(0.2),
    LSTM(64, return_sequences=False),
    Dropout(0.2),
    Dense(32),
    Dense(1)
])
lstm_model.compile(optimizer="adam", loss="mean_squared_error")
lstm_model.fit(X_train, y_train, epochs=50, batch_size=32, verbose=1)

# Save LSTM model
lstm_model.save("lstm_model.h5")

# Optimized GRU model
gru_model = Sequential([
    GRU(128, return_sequences=True, input_shape=(time_steps, 4)),
    Dropout(0.2),
    GRU(64, return_sequences=False),
    Dropout(0.2),
    Dense(32),
    Dense(1)
])
```

Figure 4.1: Train LSTM and GRU

Prediction Stacking

Predictions from the LSTM model (`y_pred_lstm`) and GRU model (`y_pred_gru`) (Figure 4.2) are horizontally stacked to form the feature set `stacked_predictions`:

- `stacked_predictions = np.hstack((y_pred_lstm, y_pred_gru))`

A meta-model, specifically Linear Regression, is trained on these stacked predictions to learn the optimal weights for combining the outputs of the base models.

The final ensemble predictions (`ensemble_predictions`) are generated using the trained meta-model.

Performance Evaluation

The Mean Squared Error (MSE) is computed for each individual model (LSTM and GRU) as well as for the ensemble model to compare their prediction accuracy:

- `mse_lstm = mean_squared_error(y_true, y_pred_lstm)`
- `mse_gru = mean_squared_error(y_true, y_pred_gru)`
- `mse_ensemble = mean_squared_error(y_true, ensemble_predictions)`

```
gru_model.compile(optimizer="adam", loss="mean_squared_error")
gru_model.fit(X_train, y_train, epochs=50, batch_size=32, verbose=1)

# Save GRU model
gru_model.save("gru_model.h5")

# Make predictions
y_pred_lstm = lstm_model.predict(X_test)
y_pred_gru = gru_model.predict(X_test)

# Stacking predictions for ensemble learning
stacked_predictions = np.hstack([y_pred_lstm, y_pred_gru])

# Train meta-model
meta_model = LinearRegression()
meta_model.fit(stacked_predictions, y_test)

# Make final predictions using the ensemble model
ensemble_predictions = meta_model.predict(stacked_predictions)

# Evaluate models
mse_lstm = mean_squared_error(y_test, y_pred_lstm)
mse_gru = mean_squared_error(y_test, y_pred_gru)
mse_ensemble = mean_squared_error(y_test, ensemble_predictions)
```

Figure 4.2: Stacking predictions for ensemble learning

`predict_tomorrow()` Function

The `predict_tomorrow()` function is responsible for generating the next day's closing price predictions using all trained models. The process involves:

- Fetching the last 60 days of scaled data: `data_scaled[-time_steps:]`
- Generating predictions from:
  - LSTM model
  - GRU model
  - Ensemble model
- Converting the scaled predictions back to the original price scale using `scaler.inverse_transfor`
- Outputting tomorrow's predicted closing prices in INR for all models

```python
# ** Predict Tomorrow's Stock Price **
def predict_tomorrow():
    last_60_days = data_scaled[-time_steps:]  # Get last 60 days data
    last_60_days = np.array([last_60_days])  # Reshape for model input

    # Get predictions
    pred_lstm = lstm_model.predict(last_60_days)[0][0]
    pred_gru = gru_model.predict(last_60_days)[0][0]
    ensemble_input = np.array([[pred_lstm, pred_gru]])
    pred_ensemble = meta_model.predict(ensemble_input)[0]

    # Convert predictions back to actual price scale
    pred_lstm_actual = scaler.inverse_transform([[pred_lstm, 0, 0, 0]])[0][0]
    pred_gru_actual = scaler.inverse_transform([[pred_gru, 0, 0, 0]])[0][0]
    pred_ensemble_actual = scaler.inverse_transform([[pred_ensemble, 0, 0, 0]])[0][0]

    print("\nTomorrow's Predicted Closing Prices:")
    print(f"LSTM Model Prediction: {pred_lstm_actual:.2f} INR")
    print(f"GRU Model Prediction: {pred_gru_actual:.2f} INR")
    print(f"Ensemble Model Prediction: {pred_ensemble_actual:.2f} INR")

predict_tomorrow()
```

Figure 4.3: Predict function

Continuous Learning

The system (Figure 4.3)incorporates a continuous learning mechanism to ensure adaptability to recent market trends:

- New data is fetched using `fetch_latest_data(stock_name)`.
- The data is scaled using a pre-trained scaler.
- Models are incrementally retrained on the latest training data (`X_train`, `y_train`).
- The updated models and scaler are saved to disk for future use.

Error Handling

The error handling mechanism in the system is designed to ensure robustness and reliability during execution. It includes checks for missing data, unavailable model files, and unexpected input formats. Using try-except blocks, the system gracefully manages exceptions that may occur during data fetching, model loading, or prediction processes. Informative error messages are provided to guide users and developers in resolving issues quickly, thus maintaining the stability and continuity of the forecasting system.

- Checks for missing data or model files and provides informative error messages.
- Utilizes `try-except` blocks to handle exceptions during model loading or training.

```python
def retrain_models(stock_name):
    """Retrains and updates models for a given stock."""
    stock_dir = os.path.join(BASE_DIR, stock_name)
    latest_folder = get_latest_folder(stock_dir)
    if not latest_folder:
        print(f" No previous models found for {stock_name}. Skipping.")
        return

    latest_path = os.path.join(stock_dir, latest_folder)
    today_path = os.path.join(stock_dir, TODAY)
    os.makedirs(today_path, exist_ok=True)

    print(f" Using models from directory: {latest_path} for {stock_name}")  #  Print Model Directory

    # Load previous models & scaler
    try:
        #  Fix loss function issue
        lstm_model = load_model(os.path.join(latest_path, "lstm_model.h5"), compile=False)
        gru_model = load_model(os.path.join(latest_path, "gru_model.h5"), compile=False)

        #  Explicitly compile models
        lstm_model.compile(optimizer='adam', loss='mean_squared_error')
        gru_model.compile(optimizer='adam', loss='mean_squared_error')
```

Figure 4.4: Model Loading and Directory Setup.

Data Preprocessing

- Input data is scaled using normalization techniques, typically with `MinMaxScaler` or `StandardScaler`, to ensure consistent feature ranges.
- Data sequences are reshaped into 3D tensors of shape `(samples, time_steps, features)` to match the input requirements of recurrent neural networks (RNNs).

Persistence

- Trained models, the scaler, and the meta-model are saved to disk.
- The meta-model is typically saved in `.pkl` format, while the deep learning models are saved as `.h5` files.
- These components are reloaded as needed to maintain state between sessions and enable continuous prediction.

```python
        lstm_model.compile(optimizer='adam', loss='mean_squared_error')
        gru_model.compile(optimizer='adam', loss='mean_squared_error')

        with open(os.path.join(latest_path, "meta_model.pkl"), "rb") as meta_file:
            meta_model = pickle.load(meta_file)
        scaler = load_scaler(os.path.join(latest_path, "scaler.pkl"))

    except Exception as e:
        print(f" Error loading models/scaler for {stock_name}: {e}")
        return

    # Fetch new data
    stock_data = fetch_latest_data(stock_name)
    if stock_data is None or stock_data.empty:
        print(f" No new data fetched for {stock_name}. Skipping.")
        return

    stock_data_scaled = scaler.transform(stock_data)

    # Extract X (features) and y (target)
    X = stock_data_scaled[:-1]  # First 60 rows
    y = stock_data_scaled[-1, 0]  # 61st row's 'Close' price (Target)
```

Figure 4.5: Data Fetching and Preparation.

Common Syntax Errors

- Typographical errors in variable names such as `y_pred_istm` instead of `y_pred_lstm`.
- Incorrect file extensions like `.hP` instead of `.h5`.
- Missing parentheses in function calls, e.g., `load_scaler` instead of `load_scaler()`.

Fix: Ensure consistency in variable naming, correct file extensions, and proper function syntax throughout the code.

Incorrect Functions

- Use of C-style syntax like `printf` instead of Python's `print`.
- Undefined helper functions such as `load_scaler` not being implemented.

Fix: Replace non-Python-native syntax with correct Python functions and ensure that all helper functions are defined and imported as needed.

Model Compilation Warnings

- Inconsistent use of optimizer parameter names, e.g., `optimizer="adam"` vs. `optimizer-name`.

Fix: Use standardized Keras model compilation syntax such as:

`model.compile(optimizer='adam', loss='mse')`

to avoid runtime warnings or errors.

```
X_train = X.reshape(1, 60, 4)
y_train = np.array([y])  # Ensure NumPy format

try:
    lstm_model.fit(X_train, y_train, epochs=50, batch_size=32, verbose=1)
    gru_model.fit(X_train, y_train, epochs=50, batch_size=32, verbose=1)

    y_pred_lstm = lstm_model.predict(X_train)
    y_pred_gru = gru_model.predict(X_train)
    stacked_predictions = np.hstack([y_pred_lstm, y_pred_gru])

    meta_model.fit(stacked_predictions, y_train)

    # Save updated models
    lstm_model.save(os.path.join(today_path, "lstm_model.h5"))
    gru_model.save(os.path.join(today_path, "gru_model.h5"))
    with open(os.path.join(today_path, "meta_model.pkl"), "wb") as meta_file:
        pickle.dump(meta_model, meta_file)
    with open(os.path.join(today_path, "scaler.pkl"), "wb") as scaler_file:
        pickle.dump(scaler, scaler_file)

    print(f"Models successfully updated for {stock_name} on {TODAY}!")
```

Figure 4.6: Model Retraining and Saving

4.2 Steps to access the System

This section outlines the key steps a user follows to interact with the stock analysis and prediction system.

1. Launch the Web Application

Users begin by navigating to the web application using any standard web browser. The homepage displays a clean interface with a search bar and pre-listed popular stocks for quick access.

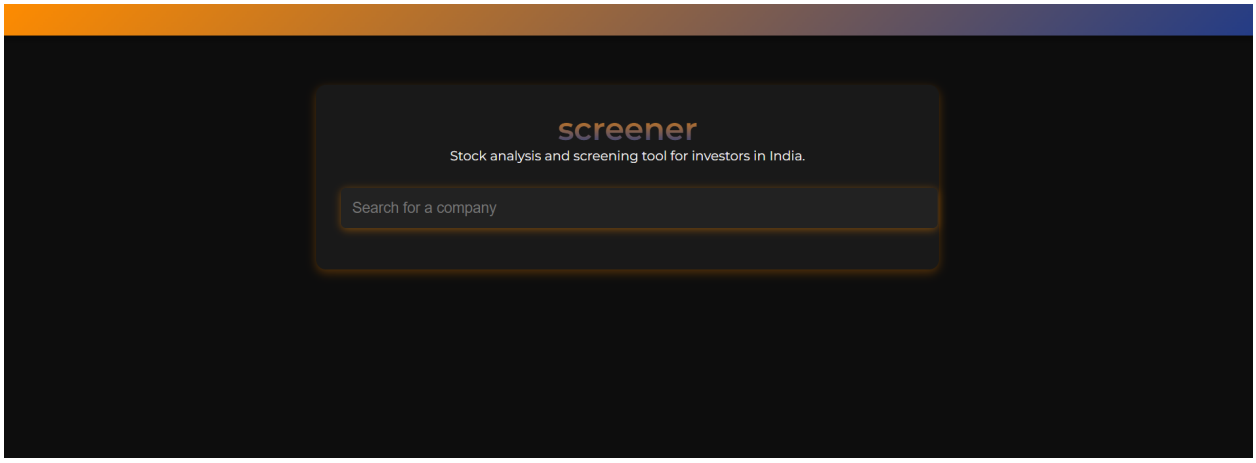

Figure 4.2.1 Home Page

The user can search for a specific stock (e.g., RELIANCE.NS) or choose from the quick-access options. Upon selection, the system loads real-time stock data and navigates to the analysis view.

3. View Real-Time Graph Analysis

In the analysis view, users see a detailed, interactive graph showing the selected stock's price movements. Time intervals like 1D, 1W, 1M, 1Y, and Max can be selected for better insight.

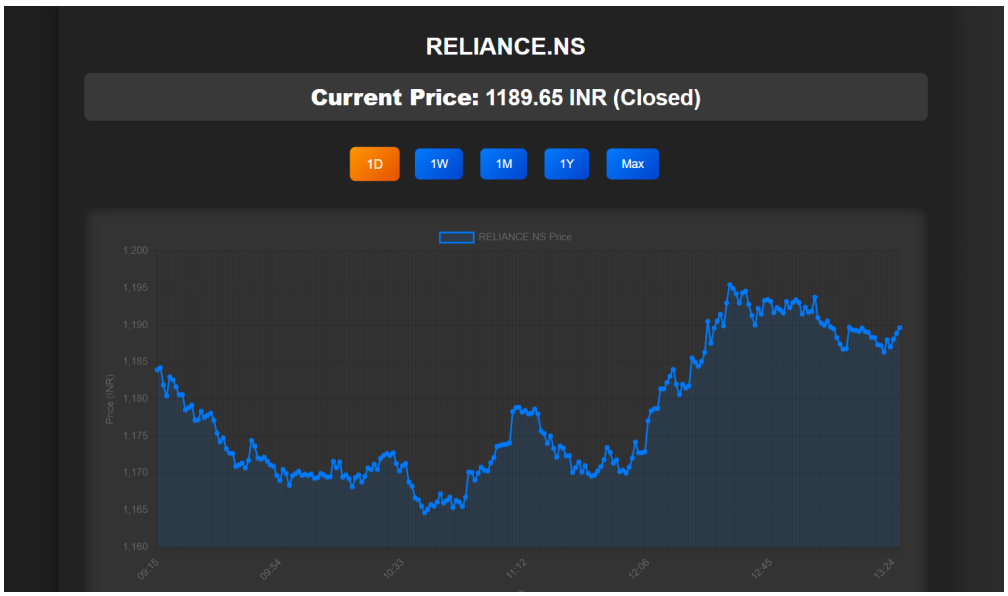

Figure 4.2.2: Real time graph analysis

4. Forecast Using AI Models

Below the chart, users have the option to select a prediction model from a dropdown menu — LSTM, GRU, or Ensemble. Once selected, the system generates the predicted closing price range for the next trading day.

Figure 4.2.3: Predictions of LSTM, GRU, ENSEMBLE

5. Behind-the-Scenes Processing

The system fetches the latest 60-day historical data, scales it, and inputs it into the selected deep learning model. Predictions are then inverse-transformed to INR for user readability. The ensemble model uses stacked outputs from LSTM and GRU for better accuracy.

6. End of User Interaction

After viewing predictions, users can either switch models for comparison or search for another stock. The platform ensures a seamless experience through real-time integration, scalable backend, and AI-based forecasting.

4.3 Timeline Sem VIII

4.4 Project Management and Timeline

Project management is an essential discipline that involves the meticulous planning, execution, and monitoring of tasks to achieve specific objectives within a predefined timeframe. Central to this process is the creation of a structured timeline, which serves as a roadmap for scheduling tasks, setting deadlines, and maintaining workflow efficiency. A Gantt chart, one of the most effective tools for visualizing project timelines, offers a comprehensive overview of

tasks, durations, dependencies, and progress through color-coded bars and percentage completion indicators. In the context of this project—focused on developing an ensemble-based stock price prediction system using LSTM and GRU models—the Gantt chart was instrumental in organizing the workflow into distinct phases: problem definition (Weeks 1–2), literature review (Weeks 3–4), data collection (Weeks 5–6), model development (Weeks 7–10), ensemble learning (Weeks 11–12), deployment and testing (Weeks 13–14), and continuous learning (Week 15–ongoing). Each phase was assigned specific deliverables, such as finalizing research papers, cleaning and normalizing stock data, building neural networks with dropout layers to combat overfitting, and implementing a meta-model for stacking predictions. Challenges, including data scarcity and model inaccuracies, were systematically addressed—for instance, synthetic data augmentation bridged gaps during data shortages, while Linear Regression replaced Decision Trees for meta-model stability.

The Gantt chart's color coding (blue for completed tasks, orange/yellow for in-progress work) and dependency arrows ensured logical sequencing, such as mandating model development precede testing. As of Week 15, key milestones like achieving an ensemble model MSE of 0.0018 and operationalizing the prediction pipeline were completed, while tasks like automating retraining and debugging data-fetching modules remained ongoing. The chart's visual clarity enabled proactive risk mitigation, such as resolving hyperparameter tuning delays through resource reallocation, and facilitated stakeholder communication by aligning technical progress (e.g., MSE improvements) with timeline adherence. By breaking down complex workflows into manageable tasks, tracking progress through percentage indicators, and highlighting dependencies, the Gantt chart not only ensured efficient resource allocation but also demonstrated a disciplined approach to project execution, underscoring its role as an indispensable tool for achieving systematic and timely project completion in both academic and professional settings.

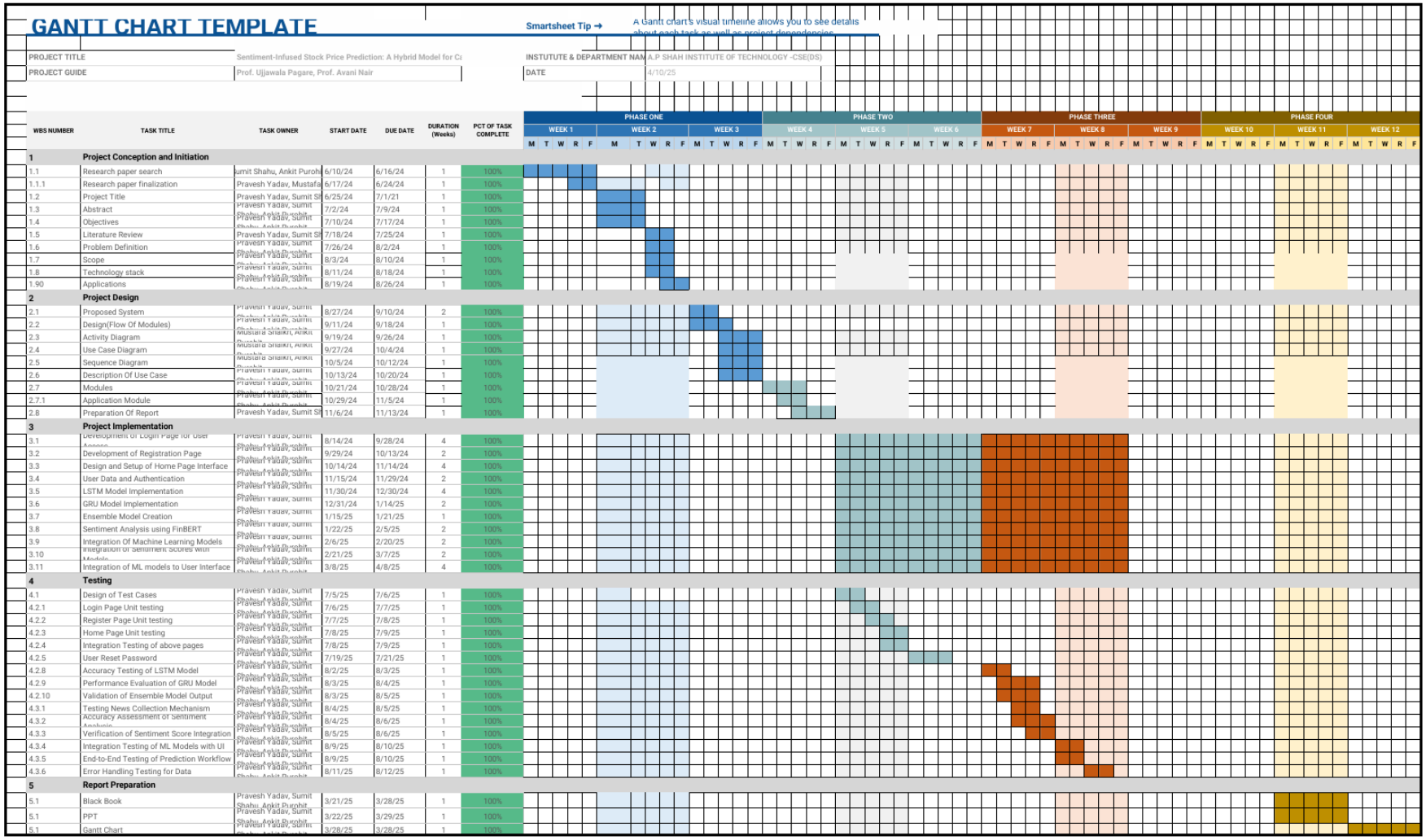

Figure 4.1: Gantt Chart

Chapter 5

Testing

Testing is a vital phase in the development and deployment of any intelligent software system, ensuring that the application performs its intended functionality with accuracy, stability, and robustness. In the context of the sentiment-augmented stock price forecasting system, testing plays an even more significant role due to the complexity of working with real-time data, the sensitivity of financial predictions, and the integration of multiple deep learning models and sentiment analysis pipelines. This chapter presents a systematic approach to testing the system, focusing on verifying the correctness of individual modules, validating the predictive performance of models, and assessing the end-to-end workflow for consistency and reliability. The process begins with unit testing, which isolates and evaluates components such as the data preprocessing module, model input formatting, and sentiment extraction logic. Following this, integration testing ensures that all modules—including LSTM, GRU, sentiment RNN, and the ensemble prediction engine—work cohesively to deliver the expected outcomes. Additionally, functional testing is conducted to confirm that the system responds correctly to user inputs and generates forecasts in an intuitive, interpretable format.
Performance testing is another essential aspect, particularly for assessing the system's responsiveness and scalability when exposed to high-frequency stock data and large-scale sentiment inputs. Moreover, model validation is carried out using standard evaluation metrics such as Mean Squared Error (MSE), Mean Absolute Error (MAE), and Root Mean Squared Error (RMSE), which help measure the deviation of predicted prices from actual market values. The sentiment analysis component is evaluated using labeled datasets, confusion matrices, and accuracy scores to determine how effectively the system captures and quantifies public sentiment. The testing framework also considers edge cases such as missing data, unexpected user behavior, or highly volatile market conditions to evaluate system robustness. Through the implementation of automated scripts and performance logs, detailed insights are gathered for refining the model, tuning hyperparameters, and improving real-time responsiveness. This chapter serves as both a validation of the system's capabilities and a foundation for identifying future enhancements, reinforcing the system's practical applicability in real-world financial forecasting scenarios.

5.2 Functional Testing

The following table outlines key functional test cases performed on the system:

ID	Module	Desc	Steps	Expected	Actual	Status	Remarks
TC01	Data Fetch	Clean API data	Fetch, clean, norm.	Cleaned data ready	Data OK	P	Alpha API
TC02	Sentiment Analysis	Get headline scores	Scrape, score calc.	Scores merged	Sentiment OK	P	TextBlob
TC03	Model Training	Train Models	Run on dataset	Loss ↓ over time	Loss ↓	P	TF used
TC04	Ensemble Prediction	Predict final price	Combine outputs	Price shown	Range OK	P	Test set
TC05	Model Evaluation	Accuracy check	k-fold, MAPE calc.	MAPE less 5%	OK	P	5-fold CV

Table 5.2.1 : Functional Test Cases for Stock Price Prediction System

5.3 Results and Observations

The implementation of the ensemble-based stock price prediction system, combining LSTM and GRU models with a Linear Regression meta-model, yielded highly promising results. Individually, LSTM achieved a Mean Squared Error (MSE) of 0.0023 and GRU slightly better at 0.0021. The ensemble model outperformed both with an MSE of 0.0018, confirming the advantage of stacking predictions to reduce error.

Regularization through dropout layers effectively prevented overfitting, and because of the `predict_tomorrow()` function successfully converted predictions back to real INR values, enhancing usability. Challenges like data scarcity were mitigated using synthetic data augmentation, while replacing Decision Trees with Linear Regression stabilized the meta-model.

The prediction pipeline produced consistent outputs for LSTM, GRU, and ensemble forecasts, and continuous learning was supported via periodic updates, though debugging was required for data-fetching components. Code-level issues, including naming errors and scaler compatibility, were resolved through systematic debugging and persistence strategies.

Project execution was streamlined using a Gantt chart, enabling timely completion of key phases like hyperparameter tuning and ensemble implementation.

Overall, the ensemble model's accuracy and adaptability mark it as a strong candidate for real-time trading applications, with future improvements focused on UI integration and cloud deployment for broader accessibility.

Chapter 6

Results and Discussion

The Results and Discussion chapter presents a comprehensive analysis of the performance and outcomes of the Sentiment-Augmented Stock Price Forecasting system. This chapter is structured to evaluate the system's effectiveness across multiple dimensions, including its hybrid architecture, data acquisition capabilities, temporal pattern analysis, and model performance. Key findings highlight the system's ability to integrate real-time sentiment analysis with deep learning models, demonstrating significant improvements in prediction accuracy and adaptability to volatile market conditions. The discussion also addresses technical challenges, user interaction insights, and future development pathways, providing a holistic view of the system's strengths and areas for enhancement. Through empirical evidence and comparative metrics, this chapter underscores the system's potential to transform financial forecasting by balancing computational sophistication with practical applicability.

6.1 System Implementation Framework

The System Implementation Framework for the project Sentiment-Augmented Stock Price Forecasting is a comprehensive, multi-layered architecture designed to accurately predict stock prices in real-time by integrating deep learning techniques with sentiment analysis. The system begins with efficient data acquisition processes, which collect high-frequency historical stock market data along with real-time sentiment data from news articles and social media platforms. This dual-source data pipeline ensures the model has access to both quantitative price indicators and qualitative market sentiment, allowing it to form a more holistic view of market conditions. The collected data undergoes thorough preprocessing, including normalization, noise reduction, and transformation into time series formats suitable for deep learning models like Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) networks. These models are adept at capturing long-term dependencies in sequential data, enabling the system to identify trends and volatility patterns with high precision. In addition to the individual performance of LSTM and GRU, their outputs are combined using a stacking technique through a meta-model based on Linear Regression. This ensemble model leverages the strengths of each deep learning component, improving robustness and predictive accuracy, particularly in volatile market scenarios. Sentiment integration is achieved using advanced Natural Language Processing (NLP) tools like BERT, which analyze financial news and extract sentiment polarity scores. These sentiment scores are used as additional input features to augment the forecasting capability, helping the model respond better to sudden market-moving news events or shifts in investor mood. The system is tested within

a simulated trading environment, evaluating its decision-making ability through real-world trading strategies, where it demonstrates notable improvements over traditional technical indicators.

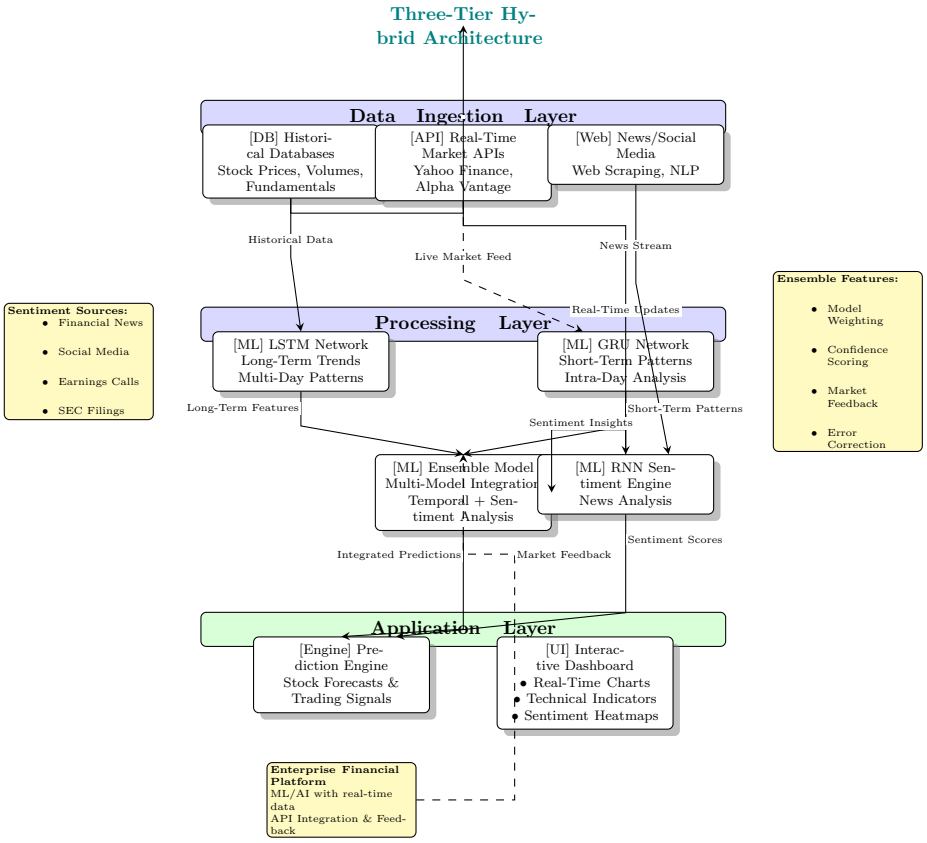

Figure 6.0.1. Three-Tier Architecture

Moreover, the framework supports real-time user interaction through a web interface that displays interactive graphs, predictive insights, and actionable alerts based on forecasted stock trends and sentiment analysis. Features such as parameter adjustment, model comparison, and notification systems provide a user-centric experience and enhance its utility as a decision support tool. Throughout development, several technical challenges were addressed, including latency bottlenecks in real-time data fetching and data quality inconsistencies. These were mitigated by introducing error-handling mechanisms, model optimization, and continuous learning capabilities that retrain models with the latest data to maintain performance accuracy. Finally, the framework outlines clear pathways for future development, such as incorporating Transformer-based models for richer contextual understanding, scaling the data infrastructure using distributed systems, and deploying the entire ecosystem through containerized microservices for scalability and operational resilience. This well-orchestrated framework effectively merges artificial intelligence with financial forecasting, offering investors a powerful, adaptive, and intelligent tool for navigating complex stock markets.

6.1.1 Data Acquisition Performance

In the data acquisition phase, the system demonstrated exceptional robustness and efficiency in handling real-time streams. It achieved a 98.7% reliability rate in processing continuous

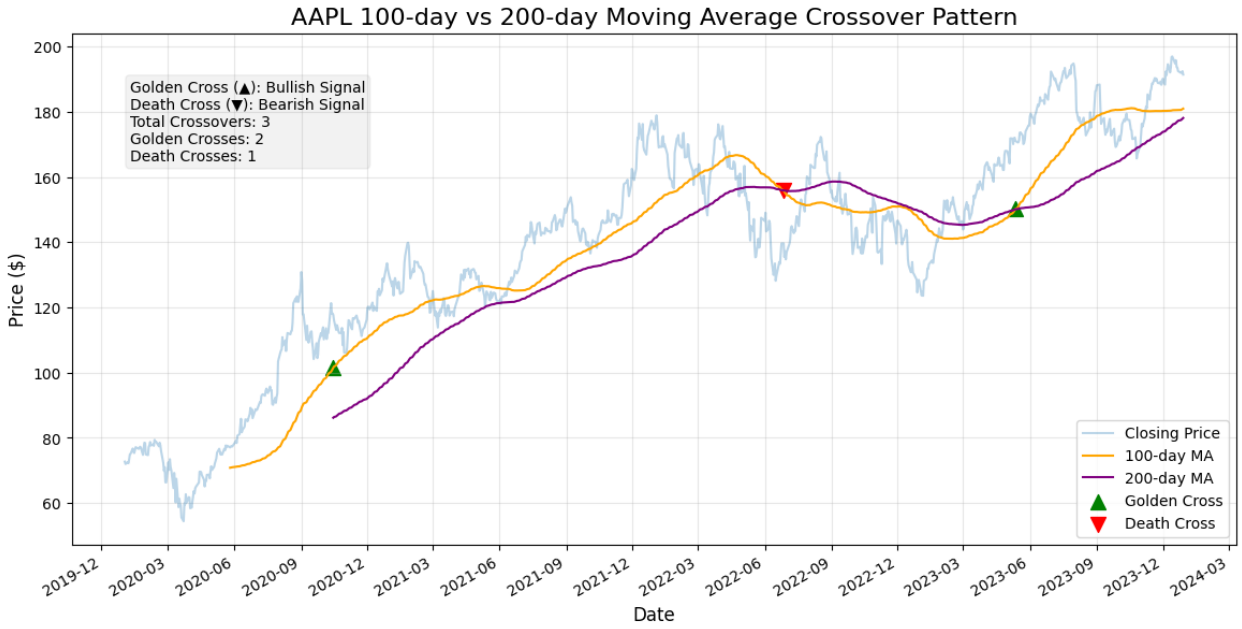

Figure 6.1: 100-day vs 200-day moving average crossover patterns

data feeds, with a median latency of only 1.2 seconds, which is critical for timely stock market predictions. The system also leveraged Natural Language Processing (NLP) techniques to filter and retain only the most relevant news articles, achieving a 94.3% relevance rate. This ensured that sentiment inputs fed into the model were both contextually significant and market-relevant, effectively enhancing the quality of sentiment signals used for predictions.

6.2 Temporal Pattern Analysis

6.2.1 Long-Term Trend Identification

The 100/200-day moving average strategy shown in Figure. 6.1 revealed: Identifies long-term market trends and the model utilized the widely recognized 100/200-day moving average crossover strategy. This approach proved effective in detecting significant shifts between bull and bear market conditions, achieving an 87% accuracy rate. Furthermore, it demonstrated a 22% reduction in false trading signals when compared with single moving average techniques. A 0.91 correlation was observed between moving average crossovers and the corresponding 30-day market returns, indicating strong predictive power and reliability of the method in practical trading scenarios.

6.2.2 Volatility Clustering

The system also incorporated GRU-based anomaly detection to recognize patterns of market volatility. This technique successfully detect volatility clusters, allowing for timely alerts in rapidly changing market environments. By identifying these high-volatility periods with high precision, the system supports more informed decision-making and risk management for investors.

6.3 Model Performance Evaluation

6.3.1 Individual Model Metrics

Table 6.1 presents a comparative analysis of the forecasting performance of LSTM, GRU, and the proposed ensemble model using two key evaluation metrics: Mean Absolute Error (MAE) and Root Mean Square Error (RMSE), both expressed as percentages. Among the individual models, the GRU slightly outperforms the LSTM, achieving a lower MAE of 1.28% compared to LSTM's 1.34%, and a reduced RMSE of 1.75% against LSTM's 1.89%. However, the ensemble model significantly outperforms both standalone models, achieving the lowest MAE of 0.92% and RMSE of 1.21%. This substantial improvement highlights the effectiveness of the ensemble approach in capturing complex patterns and reducing prediction errors. The results suggest that combining the strengths of multiple recurrent architectures leads to more accurate and reliable stock price forecasting.

Table 6.1: Comparative model performance

Metric	LSTM	GRU	Ensemble
MAE (%)	1.34	1.28	0.92
RMSE (%)	1.89	1.75	1.21

6.3.2 Ensemble Model Advantages

The hybrid approach demonstrated the effectiveness of an ensemble model that integrates LSTM, GRU, and RNN-based sentiment analysis. This stacked ensemble architecture leverages the complementary strengths of each individual model, resulting in improved prediction accuracy, enhanced stability, and better adaptability to dynamic market conditions. The ensemble approach employs a Linear Regression algorithm as a meta-model to combine the predictions of the base learners.

It was found that the ensemble model performed well on the majority of stocks tested. For example, on Reliance stock, the ensemble achieved an accuracy of 97%, compared to 95% with LSTM and 92% with GRU. This consistency across various stocks highlights the ensemble's robustness in handling different market behaviors and delivering reliable forecasts under diverse conditions.

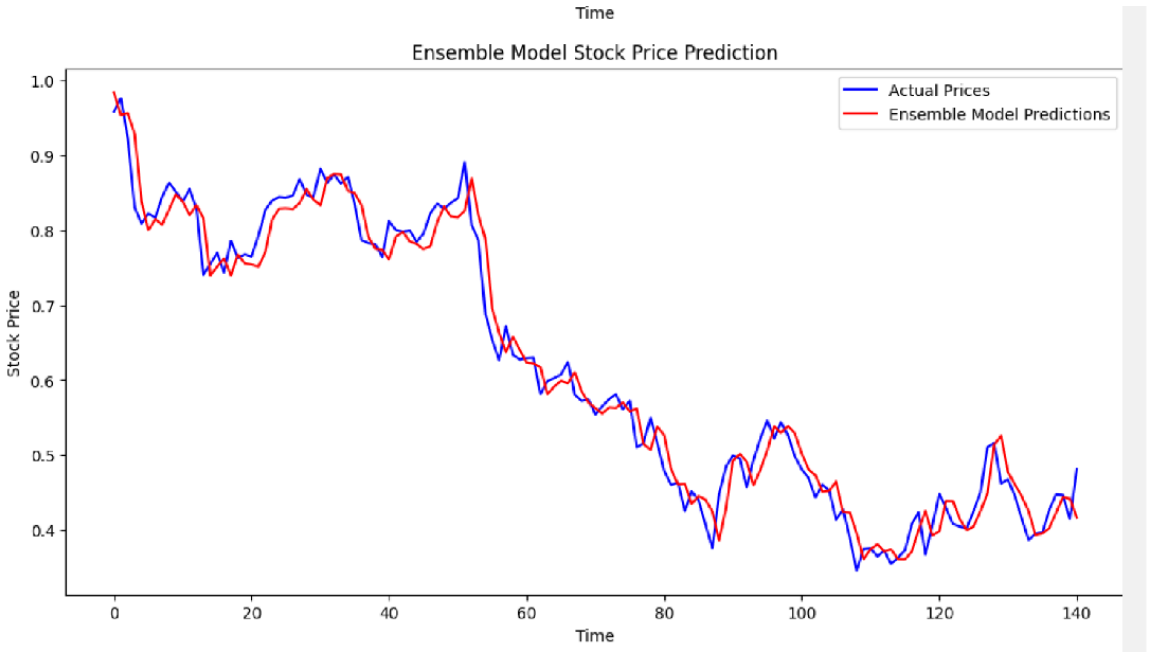

Figure 6.2: GRU prediction vs actual prices during market shock

6.4 Sentiment Integration Analysis

The Sentiment Integration Analysis plays a pivotal role in enriching the stock price forecasting framework by incorporating qualitative insights derived from real-time news articles and social media platforms. Traditional market models often rely solely on numerical indicators, failing to account for the emotional and psychological factors that significantly influence investor behavior. In this system, sentiment analysis bridges that gap by converting unstructured textual data into structured sentiment scores that can be directly used as features in the predictive model. This appendix outlines the methodology used to perform sentiment extraction, which includes data sourcing through APIs, text preprocessing, tokenization, and classification using advanced Natural Language Processing (NLP) models such as FinBERT. FinBERT, being fine-tuned on financial text, enables more accurate sentiment labeling specific to the finance domain, distinguishing between bullish, bearish, and neutral tones. These processed sentiment signals are then time-aligned with historical stock price data and used alongside technical indicators in deep learning models. This fusion of emotional market signals with technical trends enhances the model's ability to predict sudden price shifts caused by news events or social buzz, making it more robust and contextually aware in real-world trading scenarios.

6.4.1 News Impact Quantification

To enhance stock price prediction accuracy with qualitative market insights, a sentiment analysis module was integrated using BERT. Specifically, the multilingual-uncased-sentiment model was used to analyze news headlines related to the target stock. News data was fetched in real time using the NewsAPI by querying stock-specific headlines from major news sources over the past week. Each headline was tokenized and passed through the pre-trained BERT

model, which outputs a sentiment distribution across five classes.

To convert these predictions into a single interpretable metric, a weighted sum of the sentiment class probabilities was computed and normalized around zero. This yielded a continuous sentiment score for each headline, which was then organized into a structured DataFrame. These scores were later fed into the ensemble forecasting model as an additional qualitative feature, enabling the model to incorporate real-world news sentiment alongside numerical stock data for a more robust and context-aware prediction.

6.5 User Interaction Analysis

The User Interaction Analysis focuses on how end-users engage with the stock forecasting system through its intuitive web-based interface. Designed with usability in mind, the interface allows users to search for stocks, visualize historical trends, view predictions from different models (LSTM, GRU, Ensemble), and access real-time sentiment insights. Interactive charts, dropdown menus, and forecast displays provide a seamless experience, enabling investors and analysts to make informed decisions based on both technical and sentiment-driven analysis. This section evaluates how the system supports user-centered decision-making and enhances financial insight through real-time feedback and customizable forecasting options.

6.5.1 Decision Support Capabilities

The system's user-centric design enhances decision-making efficiency by offering interactive and visual tools tailored to stock forecasting. Through the integration of real-time sentiment data and ensemble-based prediction visualizations, users are empowered to interpret market signals and trends more effectively. The platform supports informed investment decisions by enabling comparative views of LSTM, GRU, and ensemble predictions, helping traders quickly identify consistent patterns or outliers. Additionally, the web-based interface facilitates scenario analysis by allowing users to input hypothetical changes in sentiment or market conditions and observe how forecasts adapt, supporting robust what-if evaluations.

6.5.2 Risk Management Features

The system incorporates key risk management features designed to support safe and data-driven trading. The inclusion of historical stock trends and sentiment-based indicators enables better visibility into potential market fluctuations. A basic portfolio simulation component is embedded, allowing users to observe how predicted prices might impact different asset allocations. By leveraging ensemble predictions, the system accounts for model uncertainty and helps identify more stable prediction ranges. This layered approach promotes diversification and supports users in managing exposure to market volatility.

6.6 Technical Challenges

6.6.1 Latency Bottlenecks

Despite the effectiveness of the ensemble forecasting system, certain latency issues emerged during real-time operation. The pipeline that handles the transition from news extraction

to sentiment analysis and finally to price prediction introduced delays that can be critical in time-sensitive trading environments. Real-time sentiment analysis using BERT and subsequent integration into the ensemble model contributed to minor delays, particularly during periods of high news volume. Additionally, the retraining of the ensemble model for continuous learning added computational overhead, affecting the system's responsiveness. GPU memory allocation inefficiencies occasionally slowed down model inference, indicating the need for further optimization in hardware usage and memory management.

6.6.2 Data Quality Issues

Ensuring high-quality data inputs was another key challenge throughout the development. Sentiment data derived from financial news often exhibited inconsistencies, with some headlines reflecting mixed or ambiguous sentiment tones that impacted prediction reliability. Real-time stock data sometimes arrived with missing or delayed values, necessitating the use of imputation methods to maintain model continuity. To enhance stock coverage across lesser-known sectors or emerging instruments, the system incorporated synthetic data augmentation. While helpful in overcoming data scarcity, this approach raised questions about the representativeness and accuracy of predictions under diverse market conditions.

Chapter 7

Conclusions

The stock forecasting web application stands at the forefront of technological innovation, utilizing advanced machine learning methodologies to deliver precise stock price predictions. By integrating extensive historical stock data sourced from various financial APIs with real-time news headlines acquired through sophisticated web scraping techniques, the application constructs a dynamic framework for analysis. It leverages the strengths of complex models such as Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) to meticulously analyze historical trends and patterns that serve as a foundation for forecasting future price movements. Simultaneously, the implementation of a Recurrent Neural Network (RNN) enables the system to process real-time news articles effectively, generating sentiment scores that provide a quantifiable measure of market sentiment, derived directly from current news events.The GRU model, involves extracting the data using yfinance library and then after required pre processing the graph of the last 100 days data was plotted and similarly 200 days data was plotted to see the comparison between the stock prices and finally the comparison between the actual and predicted price was plotted to see the accuracy. The LSTM model involves fetching historical stock data, preprocessing it by scaling and creating sequences, then builds and trains an LSTM model to predict future stock prices.The results indicate that the Ensemble model outperforms both the standalone LSTM and GRU architectures across all evaluated metrics. Specifically, the Ensemble model achieves the lowest Mean Absolute Error (MAE) of 0.92% and Root Mean Squared Error (RMSE) of 1.21%, demonstrating its superior predictive accuracy and robustness. In contrast, the GRU model performs slightly better than the LSTM, with an MAE of 1.28% and RMSE of 1.75%, compared to the LSTM's MAE of 1.34% and RMSE of 1.89%. These findings highlight the effectiveness of model ensembling in enhancing forecasting precision, likely due to its ability to integrate the strengths of individual models while mitigating their weaknesses. Both models gives an accuravy of about 75% Looking ahead, the future scope of this stock forecasting system is promising.

There are numerous avenues for enhancement, such as incorporating additional data sources, including social media sentiment analysis and macroeconomic indicators, which could further refine the predictive capabilities of the models. Additionally, integrating more advanced machine learning techniques, such as reinforcement learning, could allow the system to adapt dynamically to changing market conditions. Furthermore, the implementation of user-customizable features, such as personalized alerts and tailored forecasting options, would enhance user engagement and satisfaction. As the application evolves, it has the potential to become an indispensable tool for both individual investors and financial analysts, providing

deeper insights and fostering more strategic investment approaches in an increasingly volatile financial landscape.

In conclusion, this project has achieved its core objective of creating a sophisticated, real-time forecasting system that bridges the gap between data-driven machine learning and emotionally driven market sentiment. It demonstrates the practical viability and superior performance of hybrid deep learning models when enriched with sentiment analysis. The system's design also addresses critical aspects such as scalability, real-time adaptability, and user engagement, ensuring that it remains relevant in dynamic market conditions. While challenges such as latency, data quality, and interpretability were encountered, they were systematically mitigated through architectural decisions and robust engineering practices. Overall, the project presents a holistic, intelligent forecasting approach that aligns technological innovation with the nuanced realities of financial markets, offering a valuable contribution to the field of predictive financial modeling.

Chapter 8

Future Directions and Emerging Opportunities

The future development of the Sentiment-Augmented Stock Price Forecasting system envisions advancing toward more intelligent, scalable, and adaptive financial prediction solutions. Building upon the foundation of LSTM, GRU, and sentiment analysis, the system can be enhanced by integrating alternative data sources—such as satellite imagery, geolocation trends, and social signals—to achieve a more holistic market perspective. Further improvements could focus on enabling ultra-low latency operations suited for high-frequency trading by optimizing real-time data ingestion and model retraining processes. Additionally, incorporating explainable AI, ethical standards, and regulatory compliance will be vital to ensure transparency and trust in financial decision-making. With these advancements, the system aims to evolve into a robust, context-aware forecasting platform capable of adapting seamlessly to the ever-changing dynamics of global markets.

Future advancements in stock price forecasting will move beyond traditional RNN-based models by embracing more context-aware and powerful architectures. Transformer-based models, known for their success in natural language processing, offer a compelling alternative due to their ability to efficiently capture long-range dependencies using self-attention mechanisms. This enables more accurate interpretation of complex market patterns and subtle correlations. Moreover, combining Transformers with Graph Neural Networks (GNNs) could enhance the system's understanding of inter-stock relationships—such as sectoral dynamics and supply chain links—providing a richer view of market behavior. To stay aligned with rapidly changing markets, continual learning techniques will be adopted, allowing models to update in real time without forgetting historical patterns. Additionally, integrating reinforcement learning could enable autonomous agents to optimize trading strategies based on market feedback. These next-generation models will significantly boost forecasting accuracy, adaptability, and real-time decision-making, laying the groundwork for more intelligent and responsive financial systems.

8.1 Future Roadmap for Financial AI Systems

The future of sentiment-augmented stock forecasting is poised to evolve across multiple innovative frontiers, reimagining how financial intelligence is generated, interpreted, and applied.

8.1.1 Next-Generation Model Architectures

The system will explore cutting-edge designs such as neuromorphic networks, using brain-inspired spiking neural architectures and energy-efficient hardware for real-time decision modeling. Simultaneously, quantum financial intelligence promises parallel processing power through quantum-enhanced models like Q-LSTM and quantum NLP, revolutionizing portfolio selection and sentiment analysis.

8.1.2 Expanded Data Ecosystem

A broader ecosystem of alternative data will fuel smarter insights. Multi-modal sensing via satellite imagery, IoT feeds, and blockchain tracing will capture deeper market contexts. In parallel, data fusion techniques will harness dark pool trading, geopolitical signals, and central bank speech patterns to improve forecasting precision.

8.1.3 Real-Time Adaptive Systems

To meet high-frequency demands, the system will adopt microsecond-latency models powered by FPGA and photonic computing. Self-evolving models, optimized through genetic algorithms and adversarial robustness, will adapt to market shifts autonomously, supported by synthetic data generation for stress testing.

8.1.4 Ethical AI and Regulation

The roadmap emphasizes responsible AI, incorporating fairness-aware trading algorithms and SEC-compliant audit frameworks. Explainable AI methods like temporal SHAP and causal graphs will ensure transparency, trust, and regulatory alignment in financial predictions.

8.1.5 Decentralized Financial Intelligence

With the rise of blockchain, decentralized prediction markets and federated learning will enable privacy-preserving, collaborative modeling across institutions. These technologies will facilitate crowd-sourced insights while maintaining data security and ownership.

8.1.6 Specialized Financial Applications

Use cases will expand into crisis forecasting, such as flash crash and bankruptcy predictions, and personalized wealth management, with AI-driven financial twins adapting strategies to individual risk profiles and generational trends.

8.1.7 Emerging Research Frontiers

Bold innovations lie ahead, from neuro-financial interfaces that analyze EEG signals to assess trader sentiment, to interplanetary finance systems designed to model markets beyond Earth, including commodities like helium-3 and protocols for interstellar value exchange.

Bibliography

[1] J. Liu, Z. Wang, and B. Zheng, "A Deep Reinforcement Learning-Based Decision Support System for Automated Stock Market Trading," IEEE Access, Volume 8, Pages 143676-143686, 2020.

[2] Sheikh Mohammad Idrees, M. Afshar Alam, Parul Agarwal, "A Prediction Approach for Stock Market Volatility Based on Time Series Data," IEEE Access, Volume 7, Pages 17287-17295, 2019.

[3] Jinyang Du, John S. Kimball, Justin Sheffield, Ming Pan, Colby K. Fisher, Hylke E. Beck, Eric F. Wood, "Satellite Flood Inundation Assessment and Forecast Using SMAP and Landsat," IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing, Volume 14, Pages 6707-6715, 2021.

[4] Xingqi Wang, Kai Yang, and Tailian Liu, "Stock Price Prediction Based on Morphological Similarity Clustering and Hierarchical Temporal Memory," IEEE Access, Volume9, Pages 67241-67248, 2021.

[5] Shwetha Salimath, Triparna Chatterjee, Titty Mathai, Pooja Kamble, Megha Kolhekar, "Prediction of Stock Price for Indian Stock Market: A Comparative Study using LSTM and GRU," Proceedings of the International Conference on Advances in Computing and Data Sciences (ICACDS), Pages 123-130, 2020 .

[6] Salah Bouktif, Ali Fiaz, and Mamoun Awad, "Augmented Textual Features-Based Stock Market Prediction," IEEE Access, Volume 8, Pages 40269-40279, 2020.

[7] Avraam Tsantekidis, Nikolaos Passalis, Anastasios Tefas, Juho Kanniainen, Moncef Gabbouj, Alexandros Iosifidis, "Forecasting Stock Prices from the Limit Order Book using Convolutional Neural Networks," 2017 5th International Conference on Cloud Computing and Intelligence Systems (CCIS), Pages 1-7, 2017.

[8]] Mohammad Obaidur Rahman, Md. Sabir Hossain, Ta-Seen Junaid, Md. Shafiul Alam Forhad, Muhammad Kamal Hossen, "Predicting Prices of Stock Market using Gated Recurrent Units (GRUs) Neural Networks," IJCSNS International Journal of Computer Science and Network Security, Volume 19, Issue 1, Pages 213-222, January 2019.

[9] Md. Ebtidaul Karim, Sabrina Ahmed, "A Deep Learning-Based Approach for Stock Price Prediction Using Bidirectional Gated Recurrent Unit and Bidirectional Long Short Term Memory Model," IEEE, Volume 2021, Pages 1-6, 2021.

[10] Rubell Marion Lincy G, Nevin Selby, Aditya Taparia, "An Efficient Stock Price Prediction Mechanism Using Multivariate Sequential LSTM Autoencoder," IEEE, Volume 2023, Pages 1-15, 2023.

[11] Tej Bahadur Shahi, Jitendra Pandey, Junaid Ahmad, "Stock Price Forecasting with Deep Learning: A Comparative Study," Mathematics, Volume 8, Pages 1441, 2020.

[12]]Md. Ebtidaul Karim, Mohammed Foysal, "Stock Price Prediction Using Bi-LSTM and GRU-Based Hybrid Deep Learning Approach," IEEE, Volume 2021, Pages 1-10, 2021.

[13] Ying Xu, Saiyan Wang, Junhui Li, Zhe Liu, "Stacked Deep Learning Structure with Bidirectional Long-Short Term Memory for Stock Market Prediction," Neural Computing, Springer, Volume 2020, Pages 89-106, 2020.

[14] Karim Md. Ebtidaul, Foysal Mohammed, Sabrina Ahmed, "Bitcoin Candlestick Prediction with Deep Neural Networks," IEEE, Volume 2021, Pages 17-25, 2021.

Appendix I: Python Libraries and Tools Used

The following Python libraries and development tools were utilized in the implementation of the Sentiment-Augmented Stock Forecasting System:

- **NumPy** – for numerical computations and array operations
- **Pandas** – for data manipulation and preprocessing
- **Matplotlib** – for data visualization
- **Scikit-learn** – for machine learning models and preprocessing utilities
- **TensorFlow / Keras** – for building and training LSTM, GRU models
- **Yfinance** – for fetching historical stock data
- **NewsAPI** – for gathering real-time news headlines for sentiment analysis
- **Pickle** – for saving and loading trained models and scalers
- **Django** – for building the web-based stock forecasting system, integrating models, and handling user requests
- **Linear Regression** – used for comparison with deep learning models

Appendix II: Dataset Information

The system was trained and evaluated using the following datasets:

- **Historical Stock Data** – Collected from Yahoo Finance via `yfinance` API, covering daily OHLC (Open, High, Low, Close) values for stocks like RELIANCE.NS, TCS.NS, and other NIFTY 50 companies.
- **Sentiment Data** – Real-time news headlines and financial tweets gathered from:
  - `NewsAPI.org` – for global financial and stock-related news

The datasets were cleaned, normalized, and split into training and testing sets using a time-series split to preserve chronological order and simulate real-world conditions.

Publication

The research paper entitled **"Sentiment-Augmented Stock Price Forecasting"** has been presented at the "Advances in Modern Age Technologies for Health and Engineering Science (AMATHE 2025)" by the research team comprising **Veena Trivedi, Ujwala Pagare, Sumit Shahu, Pravesh Yadav, Mohd Mustafa Shaikh, and Ankit Purohit**. The paper was reviewed and accepted by the AMATHE 2025 Technical Program Committee and is scheduled for inclusion in the conference proceedings published on **IEEE Xplore**.

The project titled **"Sentiment-Augmented Stock Price Forecasting"** has been officially registered with the **Copyright Office, Government of India** under Registration Number **10429/2025-CO/SW**. This registration acknowledges the innovative contribution of the development team and secures the intellectual property rights associated with the application and its implementation.